

Notebook

January 13, 2025

```
[ ]: !pip install torch torchvision torchaudio --upgrade
```

```
Requirement already satisfied: torch in /usr/local/lib/python3.10/dist-packages
(2.5.1+cu121)
Requirement already satisfied: torchvision in /usr/local/lib/python3.10/dist-
packages (0.20.1+cu121)
Requirement already satisfied: torchaudio in /usr/local/lib/python3.10/dist-
packages (2.5.1+cu121)
Requirement already satisfied: filelock in /usr/local/lib/python3.10/dist-
packages (from torch) (3.16.1)
Requirement already satisfied: typing-extensions>=4.8.0 in
/usr/local/lib/python3.10/dist-packages (from torch) (4.12.2)
Requirement already satisfied: networkx in /usr/local/lib/python3.10/dist-
packages (from torch) (3.4.2)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.10/dist-packages
(from torch) (3.1.5)
Requirement already satisfied: fsspec in /usr/local/lib/python3.10/dist-packages
(from torch) (2024.10.0)
Requirement already satisfied: sympy==1.13.1 in /usr/local/lib/python3.10/dist-
packages (from torch) (1.13.1)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in
/usr/local/lib/python3.10/dist-packages (from sympy==1.13.1->torch) (1.3.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.10/dist-packages
(from torchvision) (1.26.4)
Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in
/usr/local/lib/python3.10/dist-packages (from torchvision) (11.1.0)
Requirement already satisfied: MarkupSafe>=2.0 in
/usr/local/lib/python3.10/dist-packages (from jinja2->torch) (3.0.2)
```

```
[ ]: !nvidia-smi
```

```
Mon Jan 13 14:34:17 2025
```

```
+-----+
+-----+
| NVIDIA-SMI 535.104.05                Driver Version: 535.104.05   CUDA Version:
12.2          |
|-----+-----+-----+-----+
+-----+
```

GPU Name	Persistence-M	Bus-Id	Disp.A	Volatile
Uncorr. ECC				
Fan Temp Perf	Pwr:Usage/Cap		Memory-Usage	GPU-Util
Compute M.				
MIG M.				

0 Tesla T4	Off	00000000:00:04.0	Off	
N/A 41C P8	9W / 70W	0MiB / 15360MiB		0%
Default				
N/A				

Processes:

GPU	GI	CI	PID	Type	Process name	GPU
Memory						
	ID	ID				
Usage						

No running processes found

```
[ ]: !pip install tqdm
```

Requirement already satisfied: tqdm in /usr/local/lib/python3.10/dist-packages (4.67.1)

```
[ ]: #!pip install numpy matplotlib scikit-learn tensorflow google-colab
```

```
[ ]: # Importar librerías
import os
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from tensorflow.keras.preprocessing.image import load_img, img_to_array
from google.colab import drive
```

```
# Montar Google Drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
[ ]: # Ruta base donde están los datos en Google Drive
base_path = '/content/drive/MyDrive/IA_ProyectoAgosto/datasetAffectNet/
↳AffectNet/'

# Subrutas para las carpetas de train, val y test
train_path = os.path.join(base_path, 'train')
val_path = os.path.join(base_path, 'val')
test_path = os.path.join(base_path, 'test')
```

```
[ ]: def plot_data_distribution(path):
    emotions = ['Angry', 'Disgust', 'Fear', 'Happy', 'Sad', 'Surprise']
    counts = []

    for index in range(len(emotions)):
        emotion_path = os.path.join(path, str(index))
        if os.path.exists(emotion_path): # Verifica que la ruta existe
            count = len(os.listdir(emotion_path))
            counts.append(count)
            print(f"Emotion: {emotions[index]}, Count: {count}")
        else:
            counts.append(0)
            print(f"Emotion: {emotions[index]}, Count: 0")

    # Colores diferentes para cada barra
    colors = plt.cm.tab10(np.linspace(0, 1, len(emotions)))

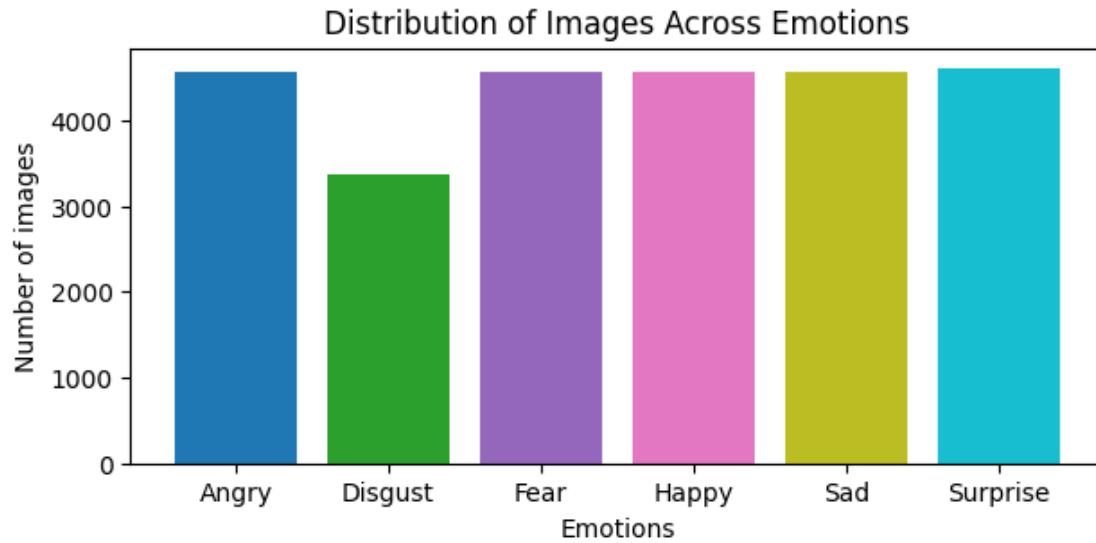
    plt.figure(figsize=(7, 3))
    plt.bar(emotions, counts, color=colors) # Usar la gama de colores definida
    plt.xlabel('Emotions')
    plt.ylabel('Number of images')
    plt.title('Distribution of Images Across Emotions')
    plt.show()

# Visualizar la distribución para train, val y test
plot_data_distribution(train_path)
plot_data_distribution(val_path)
plot_data_distribution(test_path)
```

```
Emotion: Angry, Count: 4560
Emotion: Disgust, Count: 3373
Emotion: Fear, Count: 4560
Emotion: Happy, Count: 4560
```

Emotion: Sad, Count: 4560

Emotion: Surprise, Count: 4610



Emotion: Angry, Count: 540

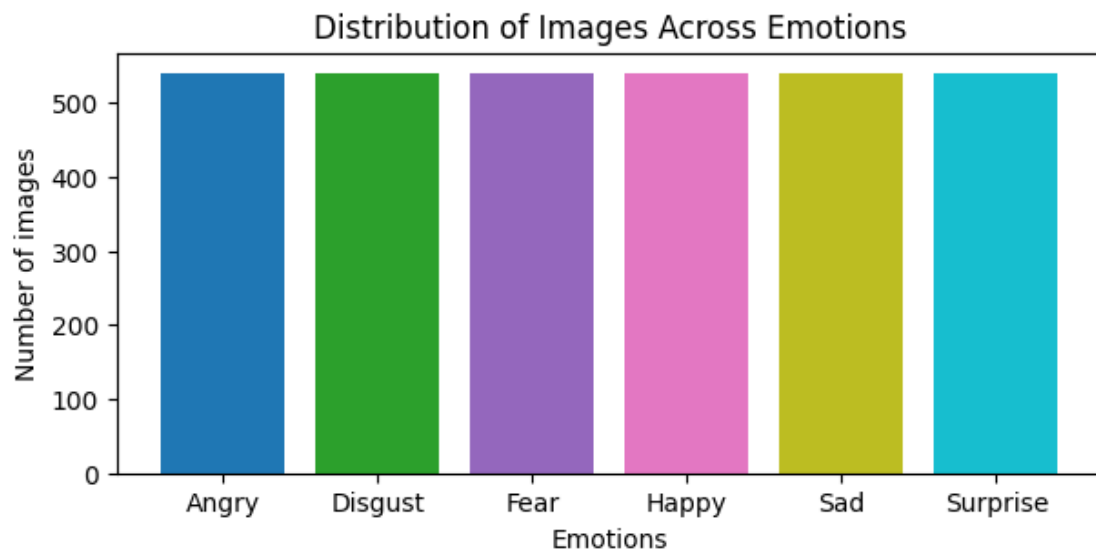
Emotion: Disgust, Count: 540

Emotion: Fear, Count: 540

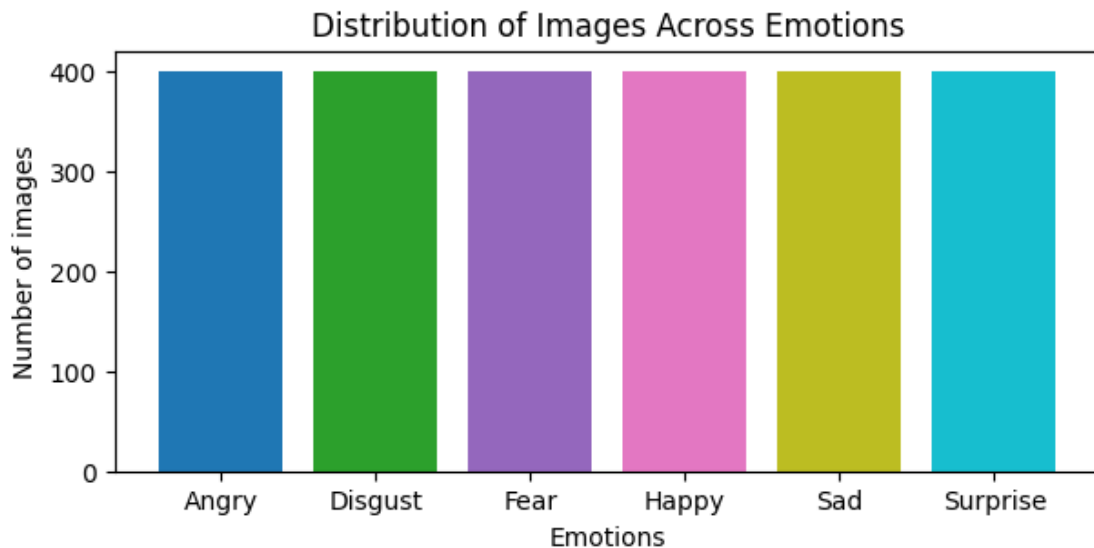
Emotion: Happy, Count: 540

Emotion: Sad, Count: 540

Emotion: Surprise, Count: 540



Emotion: Angry, Count: 400
 Emotion: Disgust, Count: 400
 Emotion: Fear, Count: 400
 Emotion: Happy, Count: 400
 Emotion: Sad, Count: 400
 Emotion: Surprise, Count: 400



```
[ ]: def show_sample_images(path, num_samples=3):
    # Definir el mapeo de los números a los nombres de las emociones
    emotion_labels = ['Angry', 'Disgust', 'Fear', 'Happy', 'Sad', 'Surprise']

    # Asignar las carpetas y ordenarlas para garantizar el orden correcto
    emotions = os.listdir(path)
    emotions.sort() # Asegura que las carpetas estén en orden numérico

    fig, axs = plt.subplots(len(emotions), num_samples, figsize=(10, 20))

    for i, emotion in enumerate(emotions):
        emotion_path = os.path.join(path, emotion)
        images = os.listdir(emotion_path)[:num_samples]
        for j, img in enumerate(images):
            img_path = os.path.join(emotion_path, img)
            image = load_img(img_path, target_size=(224, 224))
            axs[i, j].imshow(image)
            axs[i, j].axis('off')
            # Usar emotion_labels para traducir el índice numérico a etiqueta
            ↪ de emoción
```

```

        axs[i, 0].set_ylabel(emotion_labels[int(emotion)], fontsize=14,
↪rotation=0, ha='right', va='center')
        axs[i, j].set_title(emotion_labels[int(emotion)], fontsize=10) #
↪Usar emotion_labels para los títulos

plt.tight_layout(pad=3.0)
plt.show()

# Visualizar imágenes para train
show_sample_images(train_path)

```

Output hidden; open in <https://colab.research.google.com> to view.

```

[9]: #####
#      LIBRERÍAS      #
#####

import os
import time
import copy
import random
import numpy as np
import matplotlib.pyplot as plt

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
import torchvision
import torchvision.transforms as transforms
from torchvision import datasets, models
from tqdm import tqdm

# Montar Google Drive
from google.colab import drive
drive.mount('/content/drive')

#####
#      CONFIGURACIÓN      #
#####
# Directorios
base_dir = '/content/drive/MyDrive/IA_ProyectoAgosto/datasetAffectNet/AffectNet'
train_dir = os.path.join(base_dir, 'train')
val_dir   = os.path.join(base_dir, 'val')
test_dir  = os.path.join(base_dir, 'test')

# Definir la carpeta para guardar logs/estadísticas
logs_dir = '/content/drive/MyDrive/IA_ProyectoAgosto/LogsEnero/Affectnet_logs'

```

```

if not os.path.exists(logs_dir):
    os.makedirs(logs_dir)

# Hiperparámetros
num_classes = 6    # (angry=0, disgust=1, fear=2, happy=3, sad=4, surprise=5)
batch_size  = 32
num_workers = 2
epochs      = 5    # Ajustar la cantidad de épocas según sea necesario
learning_rate = 1e-4
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
print("Dispositivo usado:", device)

#####
#      TRANSFORMACIONES      #
#####
# Normalmente se usan estadísticas de ImageNet si se usan modelos preentrenados
mean = [0.485, 0.456, 0.406]
std  = [0.229, 0.224, 0.225]

# Definir transformaciones de entrenamiento y validación/test
data_transforms = {
    'train': transforms.Compose([
        transforms.Resize((224, 224)),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        transforms.Normalize(mean, std)
    ]),
    'val': transforms.Compose([
        transforms.Resize((224, 224)),
        transforms.ToTensor(),
        transforms.Normalize(mean, std)
    ]),
    'test': transforms.Compose([
        transforms.Resize((224, 224)),
        transforms.ToTensor(),
        transforms.Normalize(mean, std)
    ])
}

#####
#      DATASETS      #
#####
image_datasets = {
    'train': datasets.ImageFolder(train_dir,
    ↪transform=data_transforms['train']),
    'val':   datasets.ImageFolder(val_dir,   transform=data_transforms['val']),
    'test':  datasets.ImageFolder(test_dir,  transform=data_transforms['test'])
}

```

```

}

# DataLoaders
dataloaders = {
    'train': DataLoader(image_datasets['train'], batch_size=batch_size,
↳shuffle=True, num_workers=num_workers),
    'val': DataLoader(image_datasets['val'], batch_size=batch_size,
↳shuffle=False, num_workers=num_workers),
    'test': DataLoader(image_datasets['test'], batch_size=batch_size,
↳shuffle=False, num_workers=num_workers)
}

dataset_sizes = {
    x: len(image_datasets[x]) for x in ['train', 'val', 'test']
}

class_names = image_datasets['train'].classes
print("Clases encontradas en train:", class_names)

for split in ['train', 'val', 'test']:
    print(f"=== Conteo de imágenes en {split} ===")
    dataset = image_datasets[split]
    for class_name in dataset.classes:
        # 'dataset.class_to_idx' mapea el nombre de la clase a un índice entero
        class_idx = dataset.class_to_idx[class_name]

        # Contar cuántas muestras tienen la etiqueta 'class_idx'
        count = sum(1 for _, label in dataset.samples if label == class_idx)

        print(f"Clase: {class_name:<15} -> {count} imágenes")
    print()

#####
# FUNC. AUXILIARES #
#####
def train_model(model, criterion, optimizer, dataloaders, device, num_epochs=5,
↳logs_filename='training_log.txt'):

    import time
    import copy
    import os

```



```

"""
Entrena el modelo y guarda logs de pérdida y exactitud por época.
"""

since = time.time()

best_model_wts = copy.deepcopy(model.state_dict())
best_acc = 0.0

train_loss_history = []
val_loss_history = []
train_acc_history = []
val_acc_history = []

for epoch in range(num_epochs):
    print(f'Epoch {epoch+1}/{num_epochs}')
    print('-' * 10)

    # Cada época tiene fase de train y val
    for phase in ['train', 'val']:
        if phase == 'train':
            model.train()
        else:
            model.eval()

        running_loss = 0.0
        running_corrects = 0
        # Aquí usamos tqdm para que nos muestre progreso en cada batch.
        # 'unit="batch"' indica que la barra dirá "xx% [xx/<total>]" por
        ↪lotes.
        with tqdm(dataloaders[phase], desc=f"{phase} - epoch {epoch+1}",
        ↪unit="batch") as tepoch:
            # Iterar por los datos
            for inputs, labels in tepoch:
                inputs = inputs.to(device)
                labels = labels.to(device)

                # Poner gradientes a cero
                optimizer.zero_grad()

                # forward
                # track history solo en entrenamiento
                with torch.set_grad_enabled(phase == 'train'):
                    outputs = model(inputs)
                    _, preds = torch.max(outputs, 1)
                    loss = criterion(outputs, labels)

                # backward + optimizar solo en entrenamiento

```

```

        if phase == 'train':
            loss.backward()
            optimizer.step()

        # Estadísticas
        running_loss += loss.item() * inputs.size(0)
        running_corrects += torch.sum(preds == labels.data)

        epoch_loss = running_loss / len(dataloaders[phase].dataset)
        epoch_acc = running_corrects.double() /
↪len(dataloaders[phase].dataset)

    print(f'{phase} Loss: {epoch_loss:.4f} Acc: {epoch_acc:.4f}')

    # Guardar los resultados para graficar
    if phase == 'train':
        train_loss_history.append(epoch_loss)
        train_acc_history.append(epoch_acc.item())
    else:
        val_loss_history.append(epoch_loss)
        val_acc_history.append(epoch_acc.item())

    # Realizar deep copy del mejor modelo
    if phase == 'val' and epoch_acc > best_acc:
        best_acc = epoch_acc
        best_model_wts = copy.deepcopy(model.state_dict())

    print()

    # Guardar logs en archivo de texto
    with open(os.path.join(logs_dir, logs_filename), 'a') as f:
        f.write(f"Epoch {epoch+1}/{num_epochs}, "
                f"Train Loss: {train_loss_history[-1]:.4f}, "
                f"Train Acc: {train_acc_history[-1]:.4f}, "
                f"Test Loss: {val_loss_history[-1]:.4f}, "
                f"Test Acc: {val_acc_history[-1]:.4f}\n")

    time_elapsed = time.time() - since
    print(f'Entrenamiento completo en {time_elapsed // 60:.0f}m {time_elapsed %
↪60:.0f}s')
    print(f'Mejor Accuracy en Val: {best_acc:.4f}')

    # Cargar los mejores pesos al modelo
    model.load_state_dict(best_model_wts)

    # Retor para evaluación y para poder graficar la historia

```

```

    return model, (train_loss_history, val_loss_history, train_acc_history,
↪ val_acc_history)

def visualize_metrics(history, title_prefix="Model"):
    """
    Grafica las métricas de Loss y Accuracy a lo largo de las épocas.
    """
    train_loss, val_loss, train_acc, val_acc = history

    epochs = range(1, len(train_loss) + 1)

    plt.figure(figsize=(14, 5))

    # Loss
    plt.subplot(1, 2, 1)
    plt.plot(epochs, train_loss, 'r-', label='Train Loss')
    plt.plot(epochs, val_loss, 'b-', label='Val Loss')
    plt.title(f'{title_prefix} - Loss vs Épocas')
    plt.xlabel('Épocas')
    plt.ylabel('Loss')
    plt.legend()

    # Accuracy
    plt.subplot(1, 2, 2)
    plt.plot(epochs, train_acc, 'r-', label='Train Accuracy')
    plt.plot(epochs, val_acc, 'b-', label='Val Accuracy')
    plt.title(f'{title_prefix} - Accuracy vs Épocas')
    plt.xlabel('Épocas')
    plt.ylabel('Accuracy')
    plt.legend()

    plt.show()

def visualize_predictions(model, dataloader, device, num_images=6):
    """
    Muestra imágenes del dataset junto a su predicción.
    """
    model.eval()
    images_so_far = 0
    fig = plt.figure(figsize=(10, 10))

    with torch.no_grad():
        for i, (inputs, labels) in enumerate(dataloader):
            inputs = inputs.to(device)
            labels = labels.to(device)

            outputs = model(inputs)

```

```

_, preds = torch.max(outputs, 1)

for j in range(inputs.size(0)):
    images_so_far += 1
    ax = plt.subplot(num_images//3, 3, images_so_far)
    ax.axis('off')
    ax.set_title(f'Real: {class_names[labels[j]]}\nPred: ↪{class_names[preds[j]]}')
    inp = inputs.cpu().data[j]
    inp = inp.numpy().transpose((1, 2, 0))
    inp = std * inp + mean # desnormalizar
    inp = np.clip(inp, 0, 1)
    plt.imshow(inp)

    if images_so_far == num_images:
        return

#####
#     MODELOS (2)     #
#####
# 1. ResNet18 preentrenado
model_resnet = models.resnet18(weights='IMAGENET1K_V1') # PyTorch 2.x
# Ajustar capa final al número de clases
num_ftrs = model_resnet.fc.in_features
model_resnet.fc = nn.Linear(num_ftrs, num_classes)
model_resnet = model_resnet.to(device)

# Definir loss y optimizador
criterion = nn.CrossEntropyLoss()
optimizer_resnet = optim.Adam(model_resnet.parameters(), lr=learning_rate)

# Entrenar el modelo 1
trained_resnet, history_resnet = train_model(
    model_resnet,
    criterion,
    optimizer_resnet,
    {'train': dataloaders['train'], 'val': dataloaders['val']},
    device,
    num_epochs=epochs,
    logs_filename='resnet_training_log.txt'
)

# Graficar sus métricas
visualize_metrics(history_resnet, title_prefix="ResNet18")

# 2. VGG16 preentrenado

```

```

model_vgg = models.vgg16(weights='IMAGENET1K_V1')
# Ajustar la última capa
num_ftrs = model_vgg.classifier[6].in_features
model_vgg.classifier[6] = nn.Linear(num_ftrs, num_classes)
model_vgg = model_vgg.to(device)

optimizer_vgg = optim.Adam(model_vgg.parameters(), lr=learning_rate)

trained_vgg, history_vgg = train_model(
    model_vgg,
    criterion,
    optimizer_vgg,
    {'train': dataloaders['train'], 'val': dataloaders['val']},
    device,
    num_epochs=epochs,
    logs_filename='vgg_training_log.txt'
)

visualize_metrics(history_vgg, title_prefix="VGG16")

#####
# EVALUACIÓN EN TEST #
#####
def test_model(model, dataloader, device):
    model.eval()
    correct = 0
    total = 0
    with torch.no_grad():
        for inputs, labels in dataloader:
            inputs = inputs.to(device)
            labels = labels.to(device)
            outputs = model(inputs)
            _, predicted = torch.max(outputs.data, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()
    accuracy = 100 * correct / total
    return accuracy

resnet_test_accuracy = test_model(trained_resnet, dataloaders['test'], device)
vgg_test_accuracy    = test_model(trained_vgg, dataloaders['test'], device)

print("Test Accuracy - ResNet18:", resnet_test_accuracy)
print("Test Accuracy - VGG16:   ", vgg_test_accuracy)

#####
# EJEMPLOS DE PREDIC. #
#####

```

```
# Visualizar algunas predicciones reales vs. predichas en test
visualize_predictions(trained_resnet, dataloaders['test'], device, num_images=6)
visualize_predictions(trained_vgg, dataloaders['test'], device, num_images=6)
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

Dispositivo usado: cuda:0

Clases encontradas en train: ['0', '1', '2', '3', '4', '5']

=== Conteo de imágenes en train ===

Clase: 0	-> 4560 imágenes
Clase: 1	-> 3373 imágenes
Clase: 2	-> 4560 imágenes
Clase: 3	-> 4560 imágenes
Clase: 4	-> 4560 imágenes
Clase: 5	-> 4610 imágenes

=== Conteo de imágenes en val ===

Clase: 0	-> 540 imágenes
Clase: 1	-> 540 imágenes
Clase: 2	-> 540 imágenes
Clase: 3	-> 540 imágenes
Clase: 4	-> 540 imágenes
Clase: 5	-> 540 imágenes

=== Conteo de imágenes en test ===

Clase: 0	-> 400 imágenes
Clase: 1	-> 400 imágenes
Clase: 2	-> 400 imágenes
Clase: 3	-> 400 imágenes
Clase: 4	-> 400 imágenes
Clase: 5	-> 400 imágenes

Downloading: "https://download.pytorch.org/models/resnet18-f37072fd.pth" to
/root/.cache/torch/hub/checkpoints/resnet18-f37072fd.pth

100%| | 44.7M/44.7M [00:00<00:00, 224MB/s]

Epoch 1/5

train - epoch 1: 100%| | 820/820 [2:51:02<00:00, 12.52s/batch]

train Loss: 1.1067 Acc: 0.5702

val - epoch 1: 100%| | 102/102 [20:52<00:00, 12.28s/batch]

val Loss: 0.9639 Acc: 0.6241

Epoch 2/5

train - epoch 2: 100%| | 820/820 [02:17<00:00, 5.94batch/s]

train Loss: 0.8481 Acc: 0.6767

val - epoch 2: 100%| | 102/102 [00:16<00:00, 6.24batch/s]

val Loss: 0.9478 Acc: 0.6377

Epoch 3/5

train - epoch 3: 100%| | 820/820 [02:11<00:00, 6.25batch/s]

train Loss: 0.7209 Acc: 0.7307

val - epoch 3: 100%| | 102/102 [00:16<00:00, 6.30batch/s]

val Loss: 1.0014 Acc: 0.6336

Epoch 4/5

train - epoch 4: 100%| | 820/820 [02:13<00:00, 6.16batch/s]

train Loss: 0.6005 Acc: 0.7770

val - epoch 4: 100%| | 102/102 [00:16<00:00, 6.26batch/s]

val Loss: 1.0417 Acc: 0.6364

Epoch 5/5

train - epoch 5: 100%| | 820/820 [02:12<00:00, 6.18batch/s]

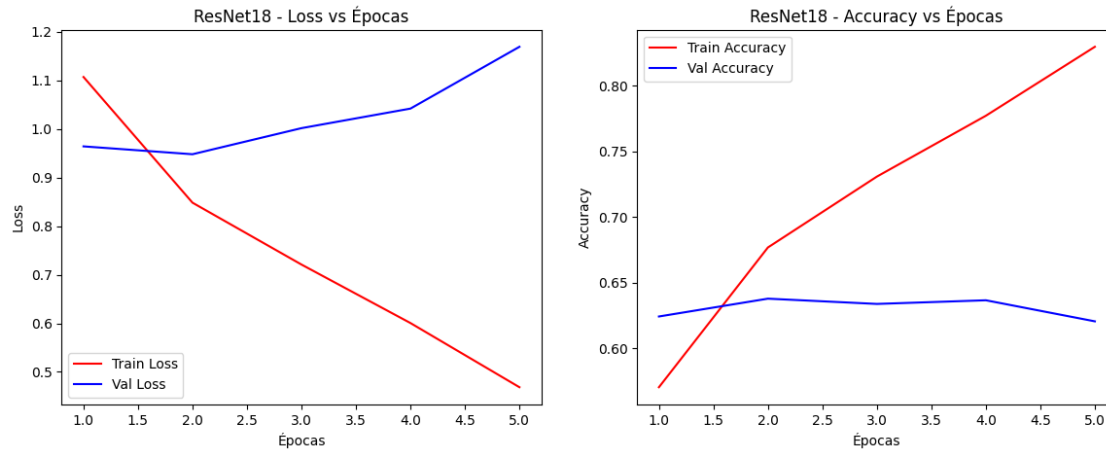
train Loss: 0.4685 Acc: 0.8295

val - epoch 5: 100%| | 102/102 [00:16<00:00, 6.26batch/s]

val Loss: 1.1688 Acc: 0.6204

Entrenamiento completo en 201m 56s

Mejor Accuracy en Val: 0.637654



Downloading: "<https://download.pytorch.org/models/vgg16-397923af.pth>" to
 /root/.cache/torch/hub/checkpoints/vgg16-397923af.pth
 100%| | 528M/528M [00:02<00:00, 240MB/s]

Epoch 1/5

```
-----
train - epoch 1: 100%| | 820/820 [06:54<00:00, 1.98batch/s]
train Loss: 1.2328 Acc: 0.5085
val - epoch 1: 100%| | 102/102 [00:17<00:00, 5.91batch/s]
val Loss: 1.0339 Acc: 0.6019
```

Epoch 2/5

```
-----
train - epoch 2: 100%| | 820/820 [06:51<00:00, 1.99batch/s]
train Loss: 0.9481 Acc: 0.6408
val - epoch 2: 100%| | 102/102 [00:17<00:00, 5.89batch/s]
val Loss: 0.9289 Acc: 0.6373
```

Epoch 3/5

```
-----
train - epoch 3: 100%| | 820/820 [06:50<00:00, 2.00batch/s]
train Loss: 0.8345 Acc: 0.6860
val - epoch 3: 100%| | 102/102 [00:17<00:00, 5.85batch/s]
val Loss: 0.9095 Acc: 0.6556
```


Epoch 4/5

train - epoch 4: 100%| | 820/820 [06:49<00:00, 2.00batch/s]

train Loss: 0.7339 Acc: 0.7267

val - epoch 4: 100%| | 102/102 [00:17<00:00, 5.71batch/s]

val Loss: 1.0325 Acc: 0.6364

Epoch 5/5

train - epoch 5: 100%| | 820/820 [06:50<00:00, 2.00batch/s]

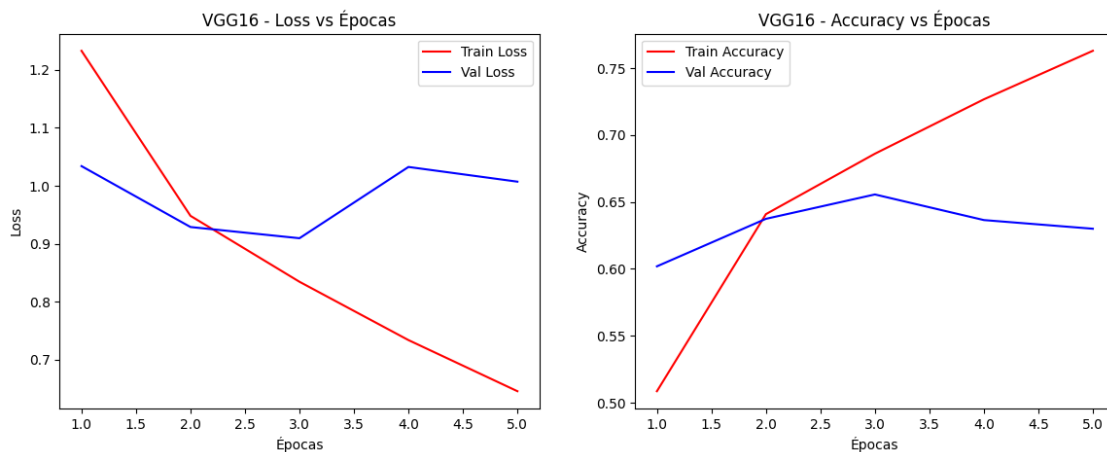
train Loss: 0.6457 Acc: 0.7629

val - epoch 5: 100%| | 102/102 [00:17<00:00, 5.69batch/s]

val Loss: 1.0071 Acc: 0.6299

Entrenamiento completo en 35m 45s

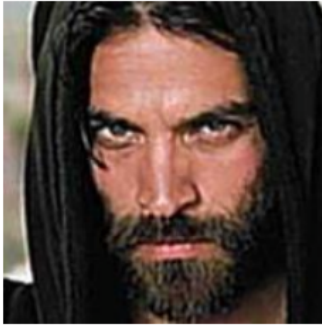
Mejor Accuracy en Val: 0.655556



Test Accuracy - ResNet18: 62.875

Test Accuracy - VGG16: 64.16666666666667

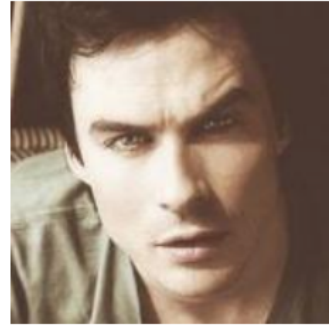
Real: 0
Pred: 0



Real: 0
Pred: 4



Real: 0
Pred: 0



Real: 0
Pred: 0



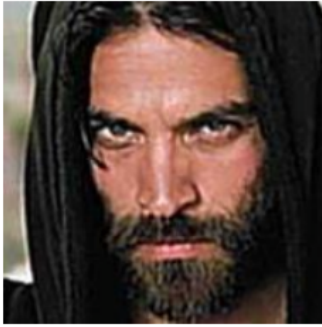
Real: 0
Pred: 5



Real: 0
Pred: 4



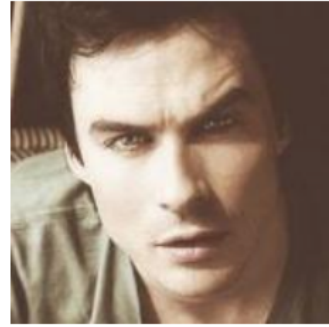
Real: 0
Pred: 0



Real: 0
Pred: 0



Real: 0
Pred: 0



Real: 0
Pred: 0



Real: 0
Pred: 0



Real: 0
Pred: 0



```
[10]: #####  
#      IMPORTAR LIBRERÍAS      #  
#####  
import os  
import time  
import copy  
import random  
import numpy as np  
import matplotlib.pyplot as plt  
  
import torch  
import torch.nn as nn  
import torch.optim as optim  
from torch.utils.data import DataLoader  
import torchvision  
import torchvision.transforms as transforms
```

```

from torchvision import datasets, models

# Barra de progreso
!pip install tqdm
from tqdm import tqdm

# Montar Google Drive
from google.colab import drive
drive.mount('/content/drive', force_remount=True)

#####
# CONFIGURACIÓN DE RUTAS Y LOGS #
#####
base_dir = '/content/drive/MyDrive/IA_ProyectoAgosto/datasetAffectNet/
↳AffectNet' # Ajusta a tu ruta
train_dir = os.path.join(base_dir, 'train')
val_dir = os.path.join(base_dir, 'val')
test_dir = os.path.join(base_dir, 'test')

# Carpeta para guardar logs
logs_dir = '/content/drive/MyDrive/IA_ProyectoAgosto/LogsEnero/Affectnet_logs'
if not os.path.exists(logs_dir):
    os.makedirs(logs_dir)

#####
# HIPERPARÁMETROS #
#####
num_classes = 6 # (angry=0, disgust=1, fear=2, happy=3, sad=4, surprise=5)
batch_size = 32
num_workers = 2
epochs = 5
learning_rate = 1e-4

# Seleccionar dispositivo
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
print("Dispositivo usado:", device)

#####
# TRANSFORMACIONES DE LAS IMÁGENES #
#####
# Estadísticas de normalización usuales de ImageNet
mean = [0.485, 0.456, 0.406]
std = [0.229, 0.224, 0.225]

data_transforms = {
    'train': transforms.Compose([
        transforms.Resize((224, 224)),

```

```

        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        transforms.Normalize(mean, std)
    ]),
    'val': transforms.Compose([
        transforms.Resize((224, 224)),
        transforms.ToTensor(),
        transforms.Normalize(mean, std)
    ]),
    'test': transforms.Compose([
        transforms.Resize((224, 224)),
        transforms.ToTensor(),
        transforms.Normalize(mean, std)
    ])
]
}

#####
#           PREPARAR DATASET           #
#####
image_datasets = {
    'train': datasets.ImageFolder(train_dir,
    ↪transform=data_transforms['train']),
    'val': datasets.ImageFolder(val_dir, transform=data_transforms['val']),
    'test': datasets.ImageFolder(test_dir, transform=data_transforms['test'])
}

dataloaders = {
    'train': DataLoader(image_datasets['train'], batch_size=batch_size,
    ↪shuffle=True, num_workers=num_workers),
    'val': DataLoader(image_datasets['val'], batch_size=batch_size,
    ↪shuffle=False, num_workers=num_workers),
    'test': DataLoader(image_datasets['test'], batch_size=batch_size,
    ↪shuffle=False, num_workers=num_workers)
}

dataset_sizes = {x: len(image_datasets[x]) for x in ['train', 'val', 'test']}

class_names = image_datasets['train'].classes
print("Clases encontradas:", class_names)

#####
#           FUNCIÓN DE ENTRENAMIENTO (con tqdm)           #
#####
def train_model(model, criterion, optimizer, dataloaders, device, num_epochs=5,
    ↪logs_filename='training_log.txt'):
    """

```

```

    Entrena el modelo y registra el mejor modelo basado en la exactitud de
    ↪ validación.
    """
    since = time.time()

    best_model_wts = copy.deepcopy(model.state_dict())
    best_acc = 0.0

    train_loss_history = []
    val_loss_history = []
    train_acc_history = []
    val_acc_history = []

    for epoch in range(num_epochs):
        print(f'Epoch {epoch+1}/{num_epochs}')
        print('-' * 10)

        for phase in ['train', 'val']:
            if phase == 'train':
                model.train()
            else:
                model.eval()

            running_loss = 0.0
            running_corrects = 0

            # Barra de progreso por batches
            with tqdm(dataloaders[phase], desc=f"{phase} - epoch {epoch+1}",
    ↪ unit="batch") as tepoch:
                for inputs, labels in tepoch:
                    inputs = inputs.to(device)
                    labels = labels.to(device)

                    # Poner gradientes a cero
                    optimizer.zero_grad()

                    # Forward
                    with torch.set_grad_enabled(phase == 'train'):
                        outputs = model(inputs)
                        _, preds = torch.max(outputs, 1)
                        loss = criterion(outputs, labels)

                    # Backward + optimizar solo en entrenamiento
                    if phase == 'train':
                        loss.backward()
                        optimizer.step()

```

```

        # Estadísticas
        running_loss += loss.item() * inputs.size(0)
        running_correcs += torch.sum(preds == labels.data)

    epoch_loss = running_loss / dataset_sizes[phase]
    epoch_acc = running_correcs.double() / dataset_sizes[phase]

    print(f'{phase} Loss: {epoch_loss:.4f} Acc: {epoch_acc:.4f}')

    if phase == 'train':
        train_loss_history.append(epoch_loss)
        train_acc_history.append(epoch_acc.item())
    else:
        val_loss_history.append(epoch_loss)
        val_acc_history.append(epoch_acc.item())

    # Guardar el mejor modelo según la métrica de validación
    if phase == 'val' and epoch_acc > best_acc:
        best_acc = epoch_acc
        best_model_wts = copy.deepcopy(model.state_dict())

    print()

    # Guardar logs en archivo
    with open(os.path.join(logs_dir, logs_filename), 'a') as f:
        f.write(f"Epoch {epoch+1}/{num_epochs}, "
                f"Train Loss: {train_loss_history[-1]:.4f}, "
                f"Train Acc: {train_acc_history[-1]:.4f}, "
                f"Val Loss: {val_loss_history[-1]:.4f}, "
                f"Val Acc: {val_acc_history[-1]:.4f}\n")

    time_elapsed = time.time() - since
    print(f'Entrenamiento completo en {time_elapsed // 60:.0f}m {time_elapsed % 60:.0f}s')
    print(f'Mejor Accuracy en Val: {best_acc:.4f}')

    # Cargar los mejores pesos
    model.load_state_dict(best_model_wts)
    return model, (train_loss_history, val_loss_history, train_acc_history,
    val_acc_history)

#####
# FUNCIONES PARA GRAFICAR MÉTRICAS #
#####
def visualize_metrics(history, title_prefix="Model"):
    train_loss, val_loss, train_acc, val_acc = history

```

```

epochs = range(1, len(train_loss) + 1)

plt.figure(figsize=(14, 5))

# Gráfica de pérdida
plt.subplot(1, 2, 1)
plt.plot(epochs, train_loss, 'r-', label='Train Loss')
plt.plot(epochs, val_loss, 'b-', label='Test Loss')
plt.title(f'{title_prefix} - Loss vs Épocas')
plt.xlabel('Épocas')
plt.ylabel('Loss')
plt.legend()

# Gráfica de exactitud
plt.subplot(1, 2, 2)
plt.plot(epochs, train_acc, 'r-', label='Train Accuracy')
plt.plot(epochs, val_acc, 'b-', label='Test Accuracy')
plt.title(f'{title_prefix} - Accuracy vs Épocas')
plt.xlabel('Épocas')
plt.ylabel('Accuracy')
plt.legend()

plt.show()

def visualize_predictions(model, dataloader, device, num_images=6):
    """
    Muestra imágenes del dataset junto a su predicción.
    """
    model.eval()
    images_so_far = 0
    fig = plt.figure(figsize=(10, 10))

    with torch.no_grad():
        for i, (inputs, labels) in enumerate(dataloader):
            inputs = inputs.to(device)
            labels = labels.to(device)

            outputs = model(inputs)
            _, preds = torch.max(outputs, 1)

            for j in range(inputs.size(0)):
                images_so_far += 1
                ax = plt.subplot(num_images//3, 3, images_so_far)
                ax.axis('off')
                ax.set_title(f'Real: {class_names[labels[j]]}\nPred: ↪{class_names[preds[j]]}')
                inp = inputs.cpu().data[j]

```



```

        inp = inp.numpy().transpose((1, 2, 0))
        # Desnormalizar
        inp = std * inp + mean
        inp = np.clip(inp, 0, 1)
        plt.imshow(inp)

    if images_so_far == num_images:
        return

#####
#   DEFINIR MODELO: MobileNetV2   #
#####
model_mobilenet = models.mobilenet_v2(weights='IMAGENET1K_V1') # PyTorch 2.x

# Ajustar la última capa (classifier) a nuestro número de clases
# Por defecto, es una secuencia: classifier = [Dropout(p=0.2, inplace=True),
↳Linear(in_features=1280, out_features=1000, bias=True)]
# Cambiar la capa final a out_features = num_classes (6)
model_mobilenet.classifier[1] = nn.Linear(model_mobilenet.classifier[1].
↳in_features, num_classes)
model_mobilenet = model_mobilenet.to(device)

# Definir función de pérdida y optimizador
criterion = nn.CrossEntropyLoss()
optimizer_mobilenet = optim.Adam(model_mobilenet.parameters(), lr=learning_rate)

#####
#   ENTRENAR MODELO MOBILE NET   #
#####
trained_mobilenet, history_mobilenet = train_model(
    model_mobilenet,
    criterion,
    optimizer_mobilenet,
    {'train': dataloaders['train'], 'val': dataloaders['val']},
    device,
    num_epochs=epochs,
    logs_filename='mobilenet_training_log.txt'
)

# Graficar métricas
visualize_metrics(history_mobilenet, title_prefix="MobileNetV2")

#####
#   EVALUAR EN EL CONJUNTO DE TEST   #
#####
def val_model(model, dataloader, device):
    model.eval()

```

```

correct = 0
total = 0
with torch.no_grad():
    for inputs, labels in dataloader:
        inputs = inputs.to(device)
        labels = labels.to(device)
        outputs = model(inputs)
        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()
accuracy = 100 * correct / total
return accuracy

mobilenet_test_accuracy = test_model(trained_mobilenet, dataloaders['test'],
    ↪device)
print("Test Accuracy - MobileNetV2:", mobilenet_test_accuracy)

#####
# EJEMPLOS DE PREDICCIONES (TETS) #
#####
visualize_predictions(trained_mobilenet, dataloaders['test'], device,
    ↪num_images=6)

```

Requirement already satisfied: tqdm in /usr/local/lib/python3.10/dist-packages (4.67.1)

Mounted at /content/drive

Dispositivo usado: cuda:0

Downloading: "https://download.pytorch.org/models/mobilenet_v2-b0353104.pth" to /root/.cache/torch/hub/checkpoints/mobilenet_v2-b0353104.pth

Clases encontradas: ['0', '1', '2', '3', '4', '5']

100%| | 13.6M/13.6M [00:00<00:00, 213MB/s]

Epoch 1/5

train - epoch 1: 100%| | 820/820 [02:49<00:00, 4.83batch/s]

train Loss: 1.1555 Acc: 0.5440

val - epoch 1: 100%| | 102/102 [00:15<00:00, 6.56batch/s]

val Loss: 0.9733 Acc: 0.6145

Epoch 2/5

train - epoch 2: 100%| | 820/820 [02:08<00:00, 6.41batch/s]

train Loss: 0.9078 Acc: 0.6542

```
val - epoch 2: 100%|      | 102/102 [00:16<00:00,  6.18batch/s]
val Loss: 0.9666 Acc: 0.6275
```

Epoch 3/5

```
-----
train - epoch 3: 100%|      | 820/820 [02:10<00:00,  6.30batch/s]
train Loss: 0.8009 Acc: 0.6956
val - epoch 3: 100%|      | 102/102 [00:15<00:00,  6.50batch/s]
val Loss: 0.9400 Acc: 0.6457
```

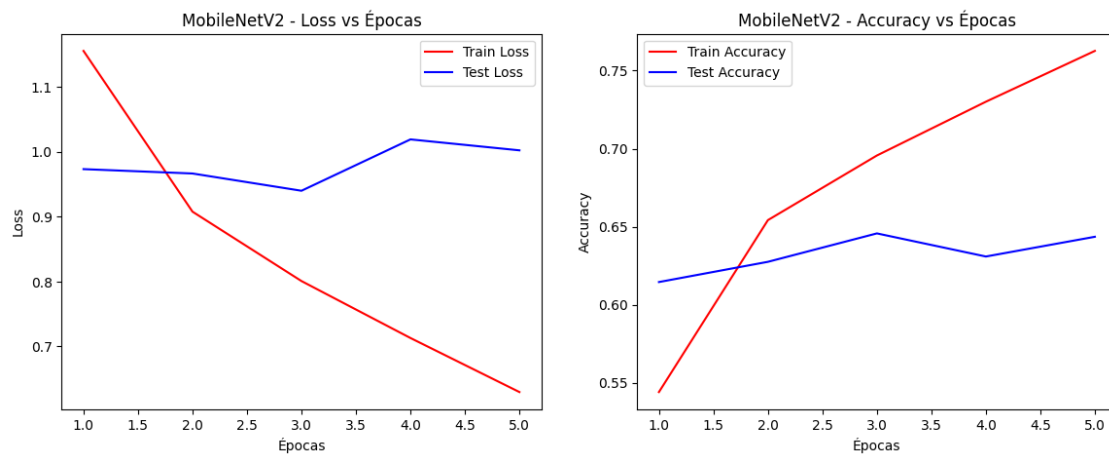
Epoch 4/5

```
-----
train - epoch 4: 100%|      | 820/820 [02:09<00:00,  6.33batch/s]
train Loss: 0.7131 Acc: 0.7301
val - epoch 4: 100%|      | 102/102 [00:15<00:00,  6.44batch/s]
val Loss: 1.0192 Acc: 0.6309
```

Epoch 5/5

```
-----
train - epoch 5: 100%|      | 820/820 [02:08<00:00,  6.39batch/s]
train Loss: 0.6298 Acc: 0.7627
val - epoch 5: 100%|      | 102/102 [00:15<00:00,  6.52batch/s]
val Loss: 1.0023 Acc: 0.6435
```

Entrenamiento completo en 12m 46s
Mejor Accuracy en Val: 0.645679



Test Accuracy - MobileNetV2: 62.291666666666664

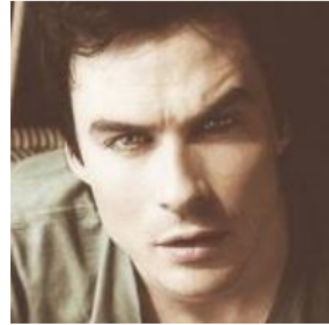
Real: 0
Pred: 0



Real: 0
Pred: 2



Real: 0
Pred: 0



Real: 0
Pred: 0



Real: 0
Pred: 0



Real: 0
Pred: 0



```
[11]: import matplotlib.pyplot as plt
import numpy as np
import torch

# 0: angry, 1: disgust, 2: fear, 3: happy,
# 4: sad, 5: surprise
emotions_list = ["angry", "disgust", "fear", "happy", "sad", "surprise"]

# mean y std de normalización (ImageNet) para deshacer la normalización al
↳mostrar
mean = [0.485, 0.456, 0.406]
std = [0.229, 0.224, 0.225]
```

```

def visualize_predictions_three_models(models, dataloader, device,
    ↪ num_images=6):
    """
    Muestra imágenes del dataset con la emoción real y la predicción de TRES
    ↪ modelos.
    models: tupla/lista con los 3 modelos (model1, model2, model3)
    dataloader: un DataLoader de prueba o validación
    device: cuda o cpu
    num_images: cuántas imágenes mostrar
    """
    model1, model2, model3 = models

    # Pasar cada modelo a eval
    model1.eval()
    model2.eval()
    model3.eval()

    images_count = 0
    fig = plt.figure(figsize=(12, 14))

    with torch.no_grad():
        for inputs, labels in dataloader:
            inputs = inputs.to(device)
            labels = labels.to(device)

            # Predicciones de cada modelo
            preds1 = model1(inputs).argmax(dim=1)
            preds2 = model2(inputs).argmax(dim=1)
            preds3 = model3(inputs).argmax(dim=1)

            # Para cada imagen del batch
            for j in range(inputs.size(0)):
                images_count += 1
                ax = plt.subplot(num_images, 1, images_count) # una fila por
                ↪ imagen, para ver las 3 preds
                ax.axis('off')

                # Desnormalizar para mostrar
                inp = inputs[j].cpu().numpy().transpose((1, 2, 0)) #
                ↪ (C,H,W) -> (H,W,C)
                inp = (inp * std) + mean
                inp = np.clip(inp, 0, 1)

                # Etiquetas reales y predichas
                real_label = emotions_list[labels[j].item()]
                pred_label_1 = emotions_list[preds1[j].item()]

```

```

pred_label_2 = emotions_list[preds2[j].item()]
pred_label_3 = emotions_list[preds3[j].item()]

# Mostrar la imagen y en el título las 3 predicciones
plt.imshow(inp)
ax.set_title(
    f"Real: {real_label}\n"
    f"ResNet: {pred_label_1}, "
    f"VGG: {pred_label_2}, "
    f"MobileNet: {pred_label_3}"
)

if images_count == num_images:
    plt.tight_layout()
    plt.show()
    return

```

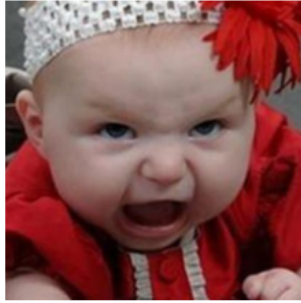
```

[12]: val_loader = DataLoader(
        image_datasets['val'],
        batch_size=batch_size,
        shuffle=False,
        num_workers=num_workers
    )

models_tuple = (trained_resnet, trained_vgg, trained_mobilenet)
visualize_predictions_three_models(models_tuple, val_loader, device,
    ↪ num_images=5)

```

Real: angry
ResNet: angry, VGG: angry, MobileNet: angry



Real: angry
ResNet: angry, VGG: angry, MobileNet: angry



Real: angry
ResNet: angry, VGG: angry, MobileNet: angry



Real: angry
ResNet: angry, VGG: angry, MobileNet: angry



Real: angry
ResNet: disgust, VGG: surprise, MobileNet: angry



```
[13]: from IPython.display import Javascript, display
from google.colab.output import eval_js
import io
import base64
from PIL import Image

def take_photo(filename='photo.jpg', quality=0.8):
    """
    Toma una foto desde la cámara en Colab y la guarda localmente como 'photo.
    ↪ jpg'.
    Retorna un objeto PIL.Image para usarlo en predicciones.
    """
    js = Javascript('''
    async function takePhoto(quality) {
        const div = document.createElement('div');
        const capture = document.createElement('button');
        capture.textContent = 'Tomar foto';
        div.appendChild(capture);

        const video = document.createElement('video');
        video.style.display = 'block';
        const stream = await navigator.mediaDevices.getUserMedia({video: true});
        document.body.appendChild(div);
        div.appendChild(video);
        video.srcObject = stream;
        await video.play();

        // Resize la salida a la resolución que quieras
        google.colab.output.setIframeHeight(document.documentElement.
        ↪ scrollHeight, true);

        // Esperar click en 'Tomar foto'
        await new Promise((resolve) => capture.onclick = resolve);

        const canvas = document.createElement('canvas');
        canvas.width = video.videoWidth;
        canvas.height = video.videoHeight;
        canvas.getContext('2d').drawImage(video, 0, 0);
        stream.getVideoTracks()[0].stop();
        const dataUrl = canvas.toDataURL('image/jpeg', quality);
        div.remove();
        return dataUrl;
    }
    ''')
```



```

display(js)
data = eval_js('takePhoto({})'.format(quality))
binary = base64.b64decode(data.split(',')[1])
with open(filename, 'wb') as f:
    f.write(binary)

return Image.open(filename)

```

```

[14]: from google.colab import files

def upload_photo():
    """
    Permite subir un archivo desde tu computadora.
    Retorna un objeto PIL.Image para usar en predicciones.
    """
    uploaded = files.upload()
    for fname in uploaded.keys():
        return Image.open(fname)
    return None

```

```

[15]: import torchvision.transforms as transforms

# Transformaciones que coinciden con tu entrenamiento (ej. ImageNet size ↵
↵ 224x224)
transform = transforms.Compose([
    transforms.Resize((224,224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std=[0.229, 0.224, 0.225])
])

def predict_emotion_three_models(models, img, device):
    """
    Dada una imagen PIL (img) y una tupla de 3 modelos,
    retorna la emoción predicha por cada modelo.
    """
    # Transformar la imagen
    img_t = transform(img).unsqueeze(0).to(device) # (1, C, H, W)

    model1, model2, model3 = models
    model1.eval()
    model2.eval()
    model3.eval()

    with torch.no_grad():
        pred1 = model1(img_t).argmax(dim=1).item()
        pred2 = model2(img_t).argmax(dim=1).item()

```

```

    pred3 = model3(img_t).argmax(dim=1).item()

    # Emoción textual
    emotion1 = emotions_list[pred1]
    emotion2 = emotions_list[pred2]
    emotion3 = emotions_list[pred3]

    return emotion1, emotion2, emotion3

```

```

[17]: # 1. Tomar una foto
img_pil = take_photo()

# Para subir archivo, usa:
# img_pil = upload_photo()

# 2. Realizar la predicción
models_tuple = (trained_resnet, trained_vgg, trained_mobilenet)
emotion_resnet, emotion_vgg, emotion_mobilenet =
    predict_emotion_three_models(models_tuple, img_pil, device)

print("Predicción ResNet:", emotion_resnet)
print("Predicción VGG:   ", emotion_vgg)
print("Predicción MobileNet:", emotion_mobilenet)

```

<IPython.core.display.Javascript object>

Predicción ResNet: happy
 Predicción VGG: surprise
 Predicción MobileNet: happy

```

[18]: import os
import time
import torch
import torchvision.transforms as transforms
import numpy as np
from PIL import Image

# lista de emociones
emotions_list = ["angry", "disgust", "fear", "happy", "sad", "surprise"]

# Estadísticas de normalización para usar modelos preentrenados en ImageNet
mean = [0.485, 0.456, 0.406]
std  = [0.229, 0.224, 0.225]

# Transformaciones para preparar la imagen
transform = transforms.Compose([
    transforms.Resize((224,224)),

```

```

        transforms.ToTensor(),
        transforms.Normalize(mean, std)
])

# Ruta donde guardar las predicciones
predictions_path = '/content/drive/MyDrive/IA_ProyectoAgosto/Enero/Predicciones'

#####
# FUNCIONES PARA GUARDAR LA FOTO EN UNA CARPETA SEGÚN (MODELO + EMOCIÓN)
#####
def ensure_dir_exists(path):
    """Crea el directorio (y subdirectorios) si no existen."""
    if not os.path.exists(path):
        os.makedirs(path)

def save_prediction_photo(img_pil, emotion_label, model_name,
    predictions_dir=predictions_path):
    """
    Guarda la imagen (PIL) en la carpeta:
    <predictions_dir>/<model_name>/<emotion_label>/ con un nombre basado en
    timestamp.
    """
    # Construir la ruta según el modelo y la emoción
    save_path = os.path.join(predictions_dir, model_name, emotion_label)
    ensure_dir_exists(save_path)

    # Generar un nombre de archivo usando timestamp
    timestamp = int(time.time())
    filename = f"prediction_{timestamp}.jpg"
    filepath = os.path.join(save_path, filename)

    # Guardar la imagen
    img_pil.save(filepath)

def predict_emotion(model, img_pil, device):
    """
    Hace la predicción de la emoción usando un modelo (PyTorch).
    Retorna la etiqueta textual de la emoción.
    """
    model.eval()
    # Aplicar las transformaciones y mover a device
    img_t = transform(img_pil).unsqueeze(0).to(device)

    with torch.no_grad():
        outputs = model(img_t)
        _, predicted_idx = torch.max(outputs, dim=1)

```

```

emotion_label = emotions_list[predicted_idx.item()]
return emotion_label

#####
# EJEMPLO DE USO: PREDICCIÓN CON 3 MODELOS Y GUARDADO #
#####
def predict_and_save_with_three_models(img_pil, models_tuple, model_names,
device):
    """
    Dada una imagen PIL (img_pil), predice la emoción con tres modelos distintos
    y guarda la foto en la carpeta de cada modelo/emoción correspondiente.

    - models_tuple: (modelA, modelB, modelC)
    - model_names: ["ResNet", "VGG", "MobileNet"] (ejemplo)
    """
    for model, m_name in zip(models_tuple, model_names):
        emotion_label = predict_emotion(model, img_pil, device)
        print(f"Modelo: {m_name} -> Emoción predicha: {emotion_label}")
        # Guardar la foto en la carpeta
        save_prediction_photo(img_pil, emotion_label, m_name)

#####
# DEMO PARA TOMAR O SUBIR UNA FOTO Y HACER PREDICCIONES CON 3 MODELOS #
#####

models_tuple = (trained_resnet, trained_vgg, trained_mobilenet)
model_names = ["ResNet", "VGG", "MobileNet"]

#####
# 1) Tomar foto desde Colab#
#####
from IPython.display import Javascript, display
from google.colab.output import eval_js
import base64
import io

def take_photo(filename='photo.jpg', quality=0.8):
    """
    Toma una foto desde la cámara en Colab y la retorna como PIL.Image.
    """
    js = Javascript('''
    async function takePhoto(quality) {
        const div = document.createElement('div');
        const capture = document.createElement('button');
        capture.textContent = 'Tomar foto';
        div.appendChild(capture);
    ''')

```

```

const video = document.createElement('video');
video.style.display = 'block';
const stream = await navigator.mediaDevices.getUserMedia({video: true});
document.body.appendChild(div);
div.appendChild(video);
video.srcObject = stream;
await video.play();

// Ajustar la altura del iframe a la vista de la cámara
google.colab.output.setIframeHeight(document.documentElement.
↳scrollHeight, true);

await new Promise((resolve) => capture.onclick = resolve);

const canvas = document.createElement('canvas');
canvas.width = video.videoWidth;
canvas.height = video.videoHeight;
canvas.getContext('2d').drawImage(video, 0, 0);
stream.getVideoTracks()[0].stop();
const dataUrl = canvas.toDataURL('image/jpeg', quality);
div.remove();
return dataUrl;
}
'''
display(js)
data = eval_js('takePhoto({})'.format(quality))
if data:
    binary = base64.b64decode(data.split(',')[1])
    with open(filename, 'wb') as f:
        f.write(binary)
    return Image.open(filename)
else:
    print("No se capturó la foto.")
    return None

# ubir una foto desde tu PC:
from google.colab import files

def upload_photo():
    """
    Sube una foto desde tu computadora y la retorna como PIL.Image.
    """
    uploaded = files.upload() # se abrirá el navegador para seleccionar
    for fname in uploaded.keys():
        return Image.open(fname)
    return None

```

```

# 2) Tomar la foto
img_pil = take_photo()

# 3) O subir la foto
img_pil = upload_photo()

# 4) Si la foto se obtuvo correctamente, predecir y guardar
if img_pil:
    predict_and_save_with_three_models(img_pil, models_tuple, model_names, device)

```

<IPython.core.display.Javascript object>

Modelo: ResNet -> Emoción predicha: surprise

Modelo: VGG -> Emoción predicha: surprise

Modelo: MobileNet -> Emoción predicha: happy

```

[19]: import pandas as pd
from sklearn.metrics import confusion_matrix, accuracy_score, recall_score, 
    <img alt="f1 score icon" data-bbox="135 395 155 410"/>f1_score
import seaborn as sns
# Obtener predicciones y etiquetas reales del dataset
y_true = []
y_pred_resnet = []
y_pred_vgg = []
y_pred_mobilenet = []

# Iterar el val dataloader
for inputs, labels in dataloaders['val']:
    inputs = inputs.to(device)
    labels = labels.to(device)

    # Obtener predicciones de cada modelo
    with torch.no_grad():
        outputs_resnet = trained_resnet(inputs)
        outputs_vgg = trained_vgg(inputs)
        outputs_mobilenet = trained_mobilenet(inputs)

    # Evaluar predicciones y etiquetas de la lista
    y_true.extend(labels.cpu().numpy())
    y_pred_resnet.extend(outputs_resnet.argmax(dim=1).cpu().numpy())
    y_pred_vgg.extend(outputs_vgg.argmax(dim=1).cpu().numpy())
    y_pred_mobilenet.extend(outputs_mobilenet.argmax(dim=1).cpu().numpy())

# Calcular las métricas para cada modelo
def calculate_metrics(y_true, y_pred, model_name, loss_history):
    accuracy = accuracy_score(y_true, y_pred)
    recall = recall_score(y_true, y_pred, average='weighted') # Usar 
    <img alt="f1 score icon" data-bbox="135 895 155 910"/>'weighted' para multiclase

```

```

    f1 = f1_score(y_true, y_pred, average='weighted')
    #loss = loss_history[-1] if loss_history else 0 # Get last loss from
    ↪history if available
    loss = loss_history[-1]

    return {"Model": model_name, "Accuracy": accuracy, "Loss": loss, "Recall":
    ↪recall, "F1-score": f1}
# Crear un DataFrame con las métricas
metrics_data = []

# Reemplazar con las variables de tus predicciones y valores reales
#metrics_data.append(calculate_metrics(y_true, y_pred_resnet, "ResNet"))
#metrics_data.append(calculate_metrics(y_true, y_pred_vgg, "VGG"))
#metrics_data.append(calculate_metrics(y_true, y_pred_mobilenet, "MobileNet"))

# Pass the loss history for each model to calculate_metrics
metrics_data.append(calculate_metrics(y_true, y_pred_resnet, "ResNet",
    ↪history_resnet[0])) # history_resnet[0] is train_loss_history
metrics_data.append(calculate_metrics(y_true, y_pred_vgg, "VGG",
    ↪history_vgg[0])) # history_vgg[0] is train_loss_history
metrics_data.append(calculate_metrics(y_true, y_pred_mobilenet, "MobileNet",
    ↪history_mobilenet[0])) # history_mobilenet[0] is train_loss_history

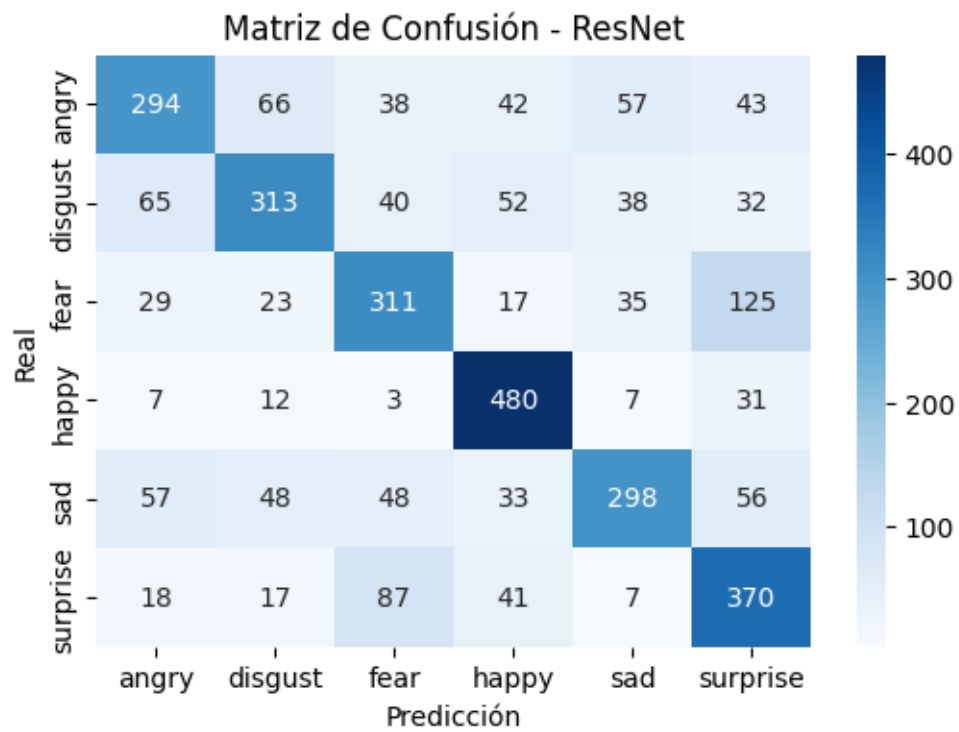
metrics_df = pd.DataFrame(metrics_data)
print(metrics_df)

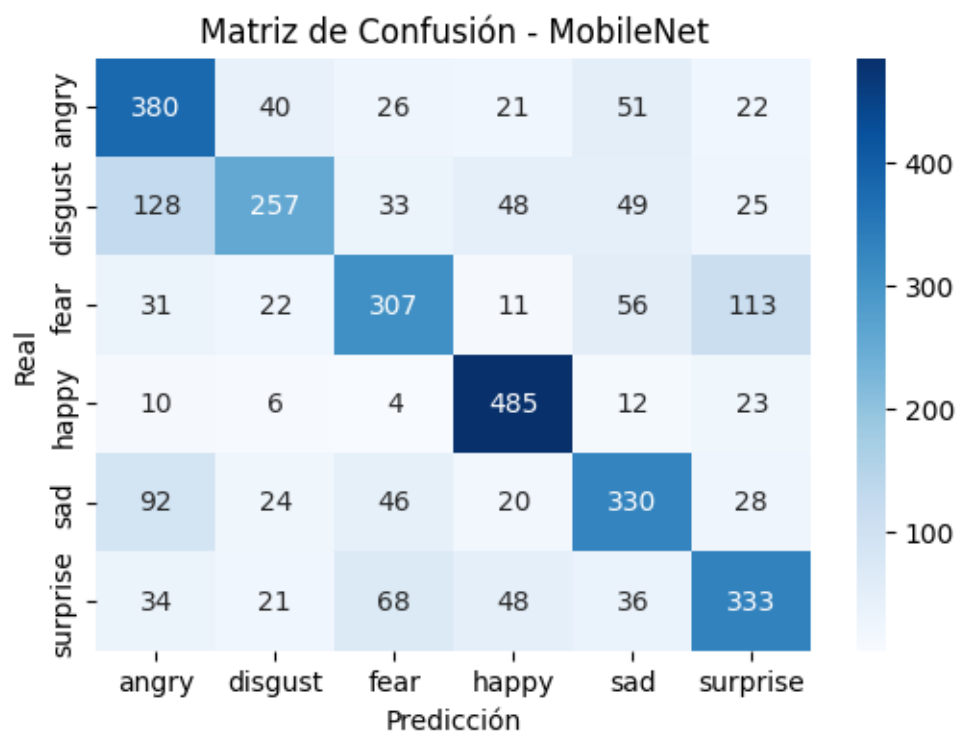
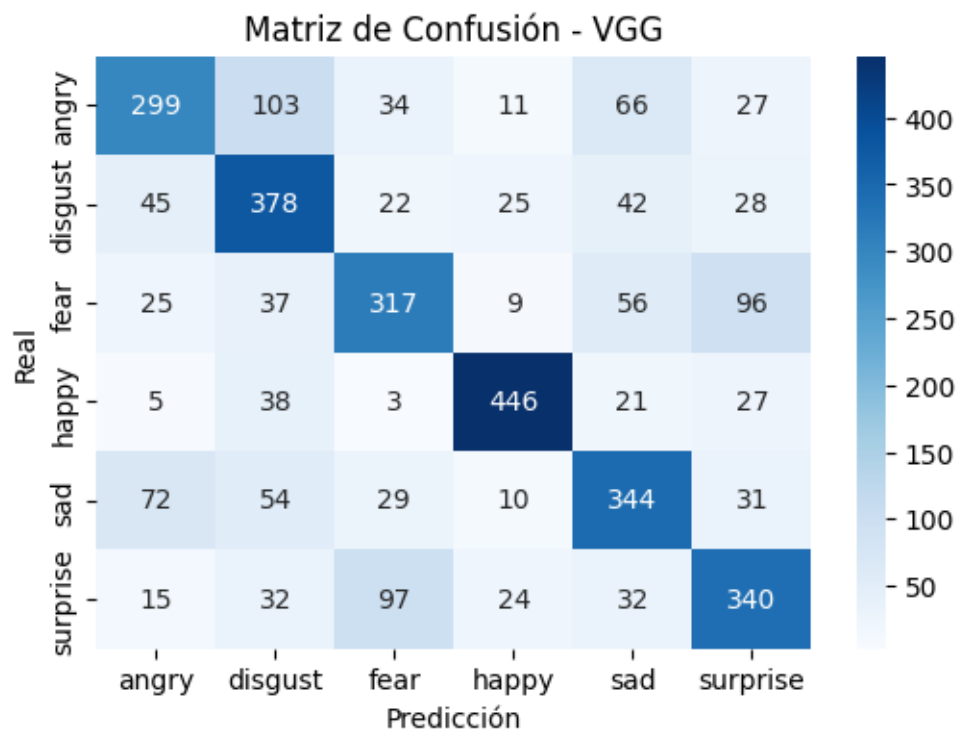
# Mostrar las matrices de confusión
def plot_confusion_matrix(y_true, y_pred, model_name):
    cm = confusion_matrix(y_true, y_pred)
    plt.figure(figsize=(6, 4))
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
                xticklabels=emotions_list, yticklabels=emotions_list)
    plt.title(f"Matriz de Confusión - {model_name}")
    plt.xlabel("Predicción")
    plt.ylabel("Real")
    plt.show()

plot_confusion_matrix(y_true, y_pred_resnet, "ResNet")
plot_confusion_matrix(y_true, y_pred_vgg, "VGG")
plot_confusion_matrix(y_true, y_pred_mobilenet, "MobileNet")

```

	Model	Accuracy	Loss	Recall	F1-score
0	ResNet	0.637654	0.468539	0.637654	0.633544
1	VGG	0.655556	0.645743	0.655556	0.655386
2	MobileNet	0.645679	0.629800	0.645679	0.640969





```
[20]: import os
import time

def ensure_dir_exists(path):
    if not os.path.exists(path):
        os.makedirs(path)

def save_model(model, model_name, epoch, log_dir='/content/drive/MyDrive/
↳IA_ProyectoAgosto/LogsEnero/Affectnet_logs'):
    """Saves a PyTorch model to a directory.

Args:
    model: The PyTorch model to save.
    model_name: The name of the model.
    epoch: The current epoch number.
    log_dir: The directory to save the model to.
    """

    timestamp = time.strftime("%Y%m%d-%H%M%S") # More precise timestamp
    model_path = os.path.join(log_dir, model_name, f"epoch_{epoch}_{timestamp}."
↳pth")
    ensure_dir_exists(os.path.dirname(model_path)) # Ensure the directory exists

    torch.save(model.state_dict(), model_path)
    print(f"Model saved to {model_path}")

# Example usage (assuming you have trained models):
# ... your existing code ...

# Example of how to use the save_model function during or after training:
# Assuming trained_resnet, trained_vgg, trained_mobilenet, and epoch are defined
# Example of how to use the save_model function during or after training:
# Assuming trained_resnet, trained_vgg, trained_mobilenet, and epoch are defined

save_model(trained_resnet, "resnet", epochs) # epochs = total number of epochs
save_model(trained_vgg, "vgg", epochs) # epochs = total number of epochs
save_model(trained_mobilenet, "mobilenet", epochs) # epochs = total number of
↳epochs
```

```
Model saved to /content/drive/MyDrive/IA_ProyectoAgosto/LogsEnero/Affectnet_logs
/resnet/epoch_5_20250113-200707.pth
Model saved to /content/drive/MyDrive/IA_ProyectoAgosto/LogsEnero/Affectnet_logs
/vgg/epoch_5_20250113-200707.pth
Model saved to /content/drive/MyDrive/IA_ProyectoAgosto/LogsEnero/Affectnet_logs
/mobilenet/epoch_5_20250113-200709.pth
```

```
[21]: def plot_six_predictions(models, img_pil, emotions_list, device):
    """Plots predictions for six emotions using three models in a single plot.
    ↪"""

    fig, axes = plt.subplots(2, 3, figsize=(12, 10)) # 2 rows, 3 columns
    fig.suptitle("Predictions for Six Emotions", fontsize=16)
    emotions = ["angry", "disgust", "fear", "happy", "sad", "surprise"]
    model_names = ["ResNet", "VGG", "MobileNet"]

    # Iterate over emotion and models
    for i in range(len(emotions)):
        row = i // 3
        col = i % 3
        ax = axes[row, col]
        emotion = emotions[i]

        img_pil_emotion = take_photo() # Capture each emotion image separately

        if img_pil_emotion:
            emotion_resnet, emotion_vgg, emotion_mobilenet = _
            ↪predict_emotion_three_models(models, img_pil_emotion, device)

            # Display image in subplot
            ax.imshow(img_pil_emotion)
            ax.set_title(f"{emotion}\nResNet:{emotion_resnet}\nVGG:
            ↪{emotion_vgg}\nMobileNet:{emotion_mobilenet}")
            ax.axis('off') # Hide axes

        plt.tight_layout(rect=[0, 0.03, 1, 0.95]) # Adjust layout to prevent _
        ↪overlap
        plt.show()

# Example usage:

#from google.colab import drive
#drive.mount('/content/drive')

#trained_resnet = '/content/drive/MyDrive/IA_ProyectoAgosto/LogsEnero/
↪Affectnet_logs/resnet/epoch_5_20250112-190621.pth'
#trained_vgg = '/content/drive/MyDrive/IA_ProyectoAgosto/LogsEnero/
↪Affectnet_logs/vgg/epoch_5_20250112-190621.pth'
#trained_mobilenet = '/content/drive/MyDrive/IA_ProyectoAgosto/LogsEnero/
↪Affectnet_logs/mobilenet/epoch_5_20250112-190623.pth'
#emotions_list = ["angry", "disgust", "fear", "happy", "sad", "surprise"]
models_tuple = (trained_resnet, trained_vgg, trained_mobilenet)
```

```
#device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
#print("Dispositivo usado:", device)

plot_six_predictions(models_tuple, None, emotions_list, device)

#["angry - enojado", "disgust - asco", "fear - miedo", "happy - feliz", "sad -
↳ triste", "surprise- sorprendido"]
```

```
<IPython.core.display.Javascript object>
<IPython.core.display.Javascript object>
<IPython.core.display.Javascript object>
<IPython.core.display.Javascript object>
<IPython.core.display.Javascript object>
<IPython.core.display.Javascript object>
```

Predictions for Six Emotions



```
[22]: !apt-get update # Update the package list
      !apt-get install texlive-xetex # Install texlive-xetex package
```

0% [Working]

Get:1 <https://cloud.r-project.org/bin/linux/ubuntu>

```

jammy-cran40/ InRelease [3,626 B]
Get:2 https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2204/x86_64
InRelease [1,581 B]
Get:3 https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2204/x86_64
Packages [1,199 kB]
Hit:4 http://archive.ubuntu.com/ubuntu jammy InRelease
Get:5 http://archive.ubuntu.com/ubuntu jammy-updates InRelease [128 kB]
Get:6 http://security.ubuntu.com/ubuntu jammy-security InRelease [129 kB]
Hit:7 https://ppa.launchpadcontent.net/deadsnakes/ppa/ubuntu jammy InRelease
Hit:8 https://ppa.launchpadcontent.net/graphics-drivers/ppa/ubuntu jammy
InRelease
Get:9 https://r2u.stat.illinois.edu/ubuntu jammy InRelease [6,555 B]
Get:10 http://archive.ubuntu.com/ubuntu jammy-backports InRelease [127 kB]
Hit:11 https://ppa.launchpadcontent.net/ubuntugis/ppa/ubuntu jammy InRelease
Get:12 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64 Packages
[1,518 kB]
Get:13 https://r2u.stat.illinois.edu/ubuntu jammy/main all Packages [8,584 kB]
Get:14 http://security.ubuntu.com/ubuntu jammy-security/multiverse amd64
Packages [45.2 kB]
Get:15 http://security.ubuntu.com/ubuntu jammy-security/main amd64 Packages
[2,560 kB]
Get:16 http://archive.ubuntu.com/ubuntu jammy-updates/restricted amd64 Packages
[3,663 kB]
Get:17 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 Packages [2,859
kB]
Get:18 http://security.ubuntu.com/ubuntu jammy-security/restricted amd64
Packages [3,527 kB]
Get:19 http://security.ubuntu.com/ubuntu jammy-security/universe amd64 Packages
[1,227 kB]
Get:20 https://r2u.stat.illinois.edu/ubuntu jammy/main amd64 Packages [2,639 kB]
Fetched 28.2 MB in 4s (6,770 kB/s)
Reading package lists... Done
W: Skipping acquire of configured file 'main/source/Sources' as repository
'https://r2u.stat.illinois.edu/ubuntu jammy InRelease' does not seem to provide
it (sources.list entry misspelt?)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  dvisvgm fonts-droid-fallback fonts-lato fonts-lmodern fonts-noto-mono fonts-
texgyre
  fonts-urw-base35 libapache-pom-java libcommons-logging-java libcommons-parent-
java
  libfontbox-java libfontenc1 libgs9 libgs9-common libidn12 libijs-0.35
libjbig2dec0 libkpathsea6
  libpdfbox-java libptexenc1 libruby3.0 libsynchronet2 libteckit0 libtexlua53
libtexluajit2 libwoff1
  libzzip-0-13 lmodern poppler-data preview-latex-style rake ruby ruby-net-

```

```

telnet ruby-rubygems
  ruby-webrick ruby-xmlrpc ruby3.0 rubygems-integration tlutils teckit tex-
common tex-gyre
  texlive-base texlive-binaries texlive-fonts-recommended texlive-latex-base
texlive-latex-extra
  texlive-latex-recommended texlive-pictures texlive-plain-generic tipa xfonts-
encodings
  xfonts-utils
Suggested packages:
  fonts-noto fonts-freefont-otf | fonts-freefont-ttf libavalon-framework-java
  libcommons-logging-java-doc libexcalibur-logkit-java liblog4j1.2-java poppler-
utils ghostscript
  fonts-japanese-mincho | fonts-ipafont-mincho fonts-japanese-gothic | fonts-
ipafont-gothic
  fonts-arphic-ukai fonts-arphic-uming fonts-nanum ri ruby-dev bundler debhelper
gv
| postscript-viewer perl-tk xpdf | pdf-viewer xzdec texlive-fonts-recommended-
doc
  texlive-latex-base-doc python3-pygments icc-profiles libfile-which-perl
  libspreadsheet-parseexcel-perl texlive-latex-extra-doc texlive-latex-
recommended-doc
  texlive-luatex texlive-pstricks dot2tex prerex texlive-pictures-doc vprerex
default-jre-headless
  tipa-doc
The following NEW packages will be installed:
  dvisvgm fonts-droid-fallback fonts-lato fonts-lmodern fonts-noto-mono fonts-
texgyre
  fonts-urw-base35 libapache-pom-java libcommons-logging-java libcommons-parent-
java
  libfontbox-java libfontenc1 libgs9 libgs9-common libidn12 libijs-0.35
libjbig2dec0 libkpathsea6
  libpdfbox-java libptexenc1 libruby3.0 libsynctex2 libteckit0 libtexlua53
libtexluajit2 libwoff1
  libzip-0-13 lmodern poppler-data preview-latex-style rake ruby ruby-net-
telnet ruby-rubygems
  ruby-webrick ruby-xmlrpc ruby3.0 rubygems-integration tlutils teckit tex-
common tex-gyre
  texlive-base texlive-binaries texlive-fonts-recommended texlive-latex-base
texlive-latex-extra
  texlive-latex-recommended texlive-pictures texlive-plain-generic texlive-xetex
tipa
  xfonts-encodings xfonts-utils
0 upgraded, 54 newly installed, 0 to remove and 51 not upgraded.
Need to get 182 MB of archives.
After this operation, 571 MB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu jammy/main amd64 fonts-droid-fallback all
1:6.0.1r16-1.1build1 [1,805 kB]
Get:2 http://archive.ubuntu.com/ubuntu jammy/main amd64 fonts-lato all 2.0-2.1

```

[2,696 kB]
Get:3 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 poppler-data all 0.4.11-1 [2,171 kB]
Get:4 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 tex-common all 6.17 [33.7 kB]
Get:5 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 fonts-urw-base35 all 20200910-1 [6,367 kB]
Get:6 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libgs9-common all 9.55.0~dfsg1-0ubuntu5.10 [752 kB]
Get:7 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libidn12 amd64 1.38-4ubuntu1 [60.0 kB]
Get:8 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 libijs-0.35 amd64 0.35-15build2 [16.5 kB]
Get:9 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 libjbig2dec0 amd64 0.19-3build2 [64.7 kB]
Get:10 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libgs9 amd64 9.55.0~dfsg1-0ubuntu5.10 [5,031 kB]
Get:11 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libkpathsea6 amd64 2021.20210626.59705-1ubuntu0.2 [60.4 kB]
Get:12 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 libwoff1 amd64 1.0.2-1build4 [45.2 kB]
Get:13 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 dvisvgm amd64 2.13.1-1 [1,221 kB]
Get:14 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 fonts-lmodern all 2.004.5-6.1 [4,532 kB]
Get:15 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 fonts-noto-mono all 20201225-1build1 [397 kB]
Get:16 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 fonts-texgyre all 20180621-3.1 [10.2 MB]
Get:17 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 libapache-pom-java all 18-1 [4,720 B]
Get:18 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 libcommons-parent-java all 43-1 [10.8 kB]
Get:19 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 libcommons-logging-java all 1.2-2 [60.3 kB]
Get:20 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 libfontenc1 amd64 1:1.1.4-1build3 [14.7 kB]
Get:21 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libptexenc1 amd64 2021.20210626.59705-1ubuntu0.2 [39.1 kB]
Get:22 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 rubygems-integration all 1.18 [5,336 B]
Get:23 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 ruby3.0 amd64 3.0.2-7ubuntu2.8 [50.1 kB]
Get:24 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 ruby-rubygems all 3.3.5-2 [228 kB]
Get:25 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 ruby amd64 1:3.0~exp1 [5,100 B]
Get:26 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 rake all 13.0.6-2 [61.7

kB]

Get:27 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 ruby-net-telnet all 0.1.1-2 [12.6 kB]
Get:28 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 ruby-webrick all 1.7.0-3ubuntu0.1 [52.1 kB]
Get:29 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 ruby-xmlrpc all 0.3.2-1ubuntu0.1 [24.9 kB]
Get:30 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libruby3.0 amd64 3.0.2-7ubuntu2.8 [5,113 kB]
Get:31 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libsyntax2 amd64 2021.20210626.59705-1ubuntu0.2 [55.6 kB]
Get:32 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 libteckit0 amd64 2.5.11+ds1-1 [421 kB]
Get:33 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libtexlua53 amd64 2021.20210626.59705-1ubuntu0.2 [120 kB]
Get:34 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libtexluajit2 amd64 2021.20210626.59705-1ubuntu0.2 [267 kB]
Get:35 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 libzip-0-13 amd64 0.13.72+dfsg.1-1.1 [27.0 kB]
Get:36 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 xfonts-encodings all 1:1.0.5-0ubuntu2 [578 kB]
Get:37 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 xfonts-utils amd64 1:7.7+6build2 [94.6 kB]
Get:38 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 lmodern all 2.004.5-6.1 [9,471 kB]
Get:39 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 preview-latex-style all 12.2-1ubuntu1 [185 kB]
Get:40 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 t1utils amd64 1.41-4build2 [61.3 kB]
Get:41 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 teckit amd64 2.5.11+ds1-1 [699 kB]
Get:42 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 tex-gyre all 20180621-3.1 [6,209 kB]
Get:43 <http://archive.ubuntu.com/ubuntu> jammy-updates/universe amd64 texlive-binaries amd64 2021.20210626.59705-1ubuntu0.2 [9,860 kB]
Get:44 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 texlive-base all 2021.20220204-1 [21.0 MB]
Get:45 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 texlive-fonts-recommended all 2021.20220204-1 [4,972 kB]
Get:46 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 texlive-latex-base all 2021.20220204-1 [1,128 kB]
Get:47 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 libfontbox-java all 1:1.8.16-2 [207 kB]
Get:48 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 libpdfbox-java all 1:1.8.16-2 [5,199 kB]
Get:49 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 texlive-latex-recommended all 2021.20220204-1 [14.4 MB]
Get:50 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 texlive-pictures


```

all 2021.20220204-1 [8,720 kB]
Get:51 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-latex-extra
all 2021.20220204-1 [13.9 MB]
Get:52 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-plain-
generic all 2021.20220204-1 [27.5 MB]
Get:53 http://archive.ubuntu.com/ubuntu jammy/universe amd64 tipa all 2:1.3-21
[2,967 kB]
Get:54 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-xetex all
2021.20220204-1 [12.4 MB]
Fetched 182 MB in 14s (13.4 MB/s)
Extracting templates from packages: 100%
Preconfiguring packages ...
Selecting previously unselected package fonts-droid-fallback.
(Reading database ... 123632 files and directories currently installed.)
Preparing to unpack .../00-fonts-droid-fallback_1%3a6.0.1r16-1.1build1_all.deb
...
Unpacking fonts-droid-fallback (1:6.0.1r16-1.1build1) ...
Selecting previously unselected package fonts-lato.
Preparing to unpack .../01-fonts-lato_2.0-2.1_all.deb ...
Unpacking fonts-lato (2.0-2.1) ...
Selecting previously unselected package poppler-data.
Preparing to unpack .../02-poppler-data_0.4.11-1_all.deb ...
Unpacking poppler-data (0.4.11-1) ...
Selecting previously unselected package tex-common.
Preparing to unpack .../03-tex-common_6.17_all.deb ...
Unpacking tex-common (6.17) ...
Selecting previously unselected package fonts-urw-base35.
Preparing to unpack .../04-fonts-urw-base35_20200910-1_all.deb ...
Unpacking fonts-urw-base35 (20200910-1) ...
Selecting previously unselected package libgs9-common.
Preparing to unpack .../05-libgs9-common_9.55.0~dfsg1-0ubuntu5.10_all.deb ...
Unpacking libgs9-common (9.55.0~dfsg1-0ubuntu5.10) ...
Selecting previously unselected package libidn12:amd64.
Preparing to unpack .../06-libidn12_1.38-4ubuntu1_amd64.deb ...
Unpacking libidn12:amd64 (1.38-4ubuntu1) ...
Selecting previously unselected package libijs-0.35:amd64.
Preparing to unpack .../07-libijs-0.35_0.35-15build2_amd64.deb ...
Unpacking libijs-0.35:amd64 (0.35-15build2) ...
Selecting previously unselected package libjbig2dec0:amd64.
Preparing to unpack .../08-libjbig2dec0_0.19-3build2_amd64.deb ...
Unpacking libjbig2dec0:amd64 (0.19-3build2) ...
Selecting previously unselected package libgs9:amd64.
Preparing to unpack .../09-libgs9_9.55.0~dfsg1-0ubuntu5.10_amd64.deb ...
Unpacking libgs9:amd64 (9.55.0~dfsg1-0ubuntu5.10) ...
Selecting previously unselected package libkpathsea6:amd64.
Preparing to unpack .../10-libkpathsea6_2021.20210626.59705-1ubuntu0.2_amd64.deb
...
Unpacking libkpathsea6:amd64 (2021.20210626.59705-1ubuntu0.2) ...

```

```

Selecting previously unselected package libwoff1:amd64.
Preparing to unpack .../11-libwoff1_1.0.2-1build4_amd64.deb ...
Unpacking libwoff1:amd64 (1.0.2-1build4) ...
Selecting previously unselected package dvisvgm.
Preparing to unpack .../12-dvisvgm_2.13.1-1_amd64.deb ...
Unpacking dvisvgm (2.13.1-1) ...
Selecting previously unselected package fonts-lmodern.
Preparing to unpack .../13-fonts-lmodern_2.004.5-6.1_all.deb ...
Unpacking fonts-lmodern (2.004.5-6.1) ...
Selecting previously unselected package fonts-noto-mono.
Preparing to unpack .../14-fonts-noto-mono_20201225-1build1_all.deb ...
Unpacking fonts-noto-mono (20201225-1build1) ...
Selecting previously unselected package fonts-texgyre.
Preparing to unpack .../15-fonts-texgyre_20180621-3.1_all.deb ...
Unpacking fonts-texgyre (20180621-3.1) ...
Selecting previously unselected package libapache-pom-java.
Preparing to unpack .../16-libapache-pom-java_18-1_all.deb ...
Unpacking libapache-pom-java (18-1) ...
Selecting previously unselected package libcommons-parent-java.
Preparing to unpack .../17-libcommons-parent-java_43-1_all.deb ...
Unpacking libcommons-parent-java (43-1) ...
Selecting previously unselected package libcommons-logging-java.
Preparing to unpack .../18-libcommons-logging-java_1.2-2_all.deb ...
Unpacking libcommons-logging-java (1.2-2) ...
Selecting previously unselected package libfontenc1:amd64.
Preparing to unpack .../19-libfontenc1_1%3a1.1.4-1build3_amd64.deb ...
Unpacking libfontenc1:amd64 (1:1.1.4-1build3) ...
Selecting previously unselected package libptexenc1:amd64.
Preparing to unpack .../20-libptexenc1_2021.20210626.59705-1ubuntu0.2_amd64.deb
...
Unpacking libptexenc1:amd64 (2021.20210626.59705-1ubuntu0.2) ...
Selecting previously unselected package rubygems-integration.
Preparing to unpack .../21-rubygems-integration_1.18_all.deb ...
Unpacking rubygems-integration (1.18) ...
Selecting previously unselected package ruby3.0.
Preparing to unpack .../22-ruby3.0_3.0.2-7ubuntu2.8_amd64.deb ...
Unpacking ruby3.0 (3.0.2-7ubuntu2.8) ...
Selecting previously unselected package ruby-rubygems.
Preparing to unpack .../23-ruby-rubygems_3.3.5-2_all.deb ...
Unpacking ruby-rubygems (3.3.5-2) ...
Selecting previously unselected package ruby.
Preparing to unpack .../24-ruby_1%3a3.0~exp1_amd64.deb ...
Unpacking ruby (1:3.0~exp1) ...
Selecting previously unselected package rake.
Preparing to unpack .../25-rake_13.0.6-2_all.deb ...
Unpacking rake (13.0.6-2) ...
Selecting previously unselected package ruby-net-telnet.
Preparing to unpack .../26-ruby-net-telnet_0.1.1-2_all.deb ...

```

```

Unpacking ruby-net-telnet (0.1.1-2) ...
Selecting previously unselected package ruby-webrick.
Preparing to unpack .../27-ruby-webrick_1.7.0-3ubuntu0.1_all.deb ...
Unpacking ruby-webrick (1.7.0-3ubuntu0.1) ...
Selecting previously unselected package ruby-xmlrpc.
Preparing to unpack .../28-ruby-xmlrpc_0.3.2-1ubuntu0.1_all.deb ...
Unpacking ruby-xmlrpc (0.3.2-1ubuntu0.1) ...
Selecting previously unselected package libruby3.0:amd64.
Preparing to unpack .../29-libruby3.0_3.0.2-7ubuntu2.8_amd64.deb ...
Unpacking libruby3.0:amd64 (3.0.2-7ubuntu2.8) ...
Selecting previously unselected package libsyntax2:amd64.
Preparing to unpack .../30-libsyntax2_2021.20210626.59705-1ubuntu0.2_amd64.deb
...
Unpacking libsyntax2:amd64 (2021.20210626.59705-1ubuntu0.2) ...
Selecting previously unselected package libteckit0:amd64.
Preparing to unpack .../31-libteckit0_2.5.11+ds1-1_amd64.deb ...
Unpacking libteckit0:amd64 (2.5.11+ds1-1) ...
Selecting previously unselected package libtexlua53:amd64.
Preparing to unpack .../32-libtexlua53_2021.20210626.59705-1ubuntu0.2_amd64.deb
...
Unpacking libtexlua53:amd64 (2021.20210626.59705-1ubuntu0.2) ...
Selecting previously unselected package libtexluaajit2:amd64.
Preparing to unpack
.../33-libtexluaajit2_2021.20210626.59705-1ubuntu0.2_amd64.deb ...
Unpacking libtexluaajit2:amd64 (2021.20210626.59705-1ubuntu0.2) ...
Selecting previously unselected package libzip-0-13:amd64.
Preparing to unpack .../34-libzip-0-13_0.13.72+dfsg.1-1.1_amd64.deb ...
Unpacking libzip-0-13:amd64 (0.13.72+dfsg.1-1.1) ...
Selecting previously unselected package xfonts-encodings.
Preparing to unpack .../35-xfonts-encodings_1%3a1.0.5-0ubuntu2_all.deb ...
Unpacking xfonts-encodings (1:1.0.5-0ubuntu2) ...
Selecting previously unselected package xfonts-utils.
Preparing to unpack .../36-xfonts-utils_1%3a7.7+6build2_amd64.deb ...
Unpacking xfonts-utils (1:7.7+6build2) ...
Selecting previously unselected package lmodern.
Preparing to unpack .../37-lmodern_2.004.5-6.1_all.deb ...
Unpacking lmodern (2.004.5-6.1) ...
Selecting previously unselected package preview-latex-style.
Preparing to unpack .../38-preview-latex-style_12.2-1ubuntu1_all.deb ...
Unpacking preview-latex-style (12.2-1ubuntu1) ...
Selecting previously unselected package t1utils.
Preparing to unpack .../39-t1utils_1.41-4build2_amd64.deb ...
Unpacking t1utils (1.41-4build2) ...
Selecting previously unselected package teckit.
Preparing to unpack .../40-teckit_2.5.11+ds1-1_amd64.deb ...
Unpacking teckit (2.5.11+ds1-1) ...
Selecting previously unselected package tex-gyre.
Preparing to unpack .../41-tex-gyre_20180621-3.1_all.deb ...

```

```

Unpacking tex-gyre (20180621-3.1) ...
Selecting previously unselected package texlive-binaries.
Preparing to unpack .../42-texlive-
binaries_2021.20210626.59705-1ubuntu0.2_amd64.deb ...
Unpacking texlive-binaries (2021.20210626.59705-1ubuntu0.2) ...
Selecting previously unselected package texlive-base.
Preparing to unpack .../43-texlive-base_2021.20220204-1_all.deb ...
Unpacking texlive-base (2021.20220204-1) ...
Selecting previously unselected package texlive-fonts-recommended.
Preparing to unpack .../44-texlive-fonts-recommended_2021.20220204-1_all.deb ...
Unpacking texlive-fonts-recommended (2021.20220204-1) ...
Selecting previously unselected package texlive-latex-base.
Preparing to unpack .../45-texlive-latex-base_2021.20220204-1_all.deb ...
Unpacking texlive-latex-base (2021.20220204-1) ...
Selecting previously unselected package libfontbox-java.
Preparing to unpack .../46-libfontbox-java_1%3a1.8.16-2_all.deb ...
Unpacking libfontbox-java (1:1.8.16-2) ...
Selecting previously unselected package libpdfbox-java.
Preparing to unpack .../47-libpdfbox-java_1%3a1.8.16-2_all.deb ...
Unpacking libpdfbox-java (1:1.8.16-2) ...
Selecting previously unselected package texlive-latex-recommended.
Preparing to unpack .../48-texlive-latex-recommended_2021.20220204-1_all.deb ...
Unpacking texlive-latex-recommended (2021.20220204-1) ...
Selecting previously unselected package texlive-pictures.
Preparing to unpack .../49-texlive-pictures_2021.20220204-1_all.deb ...
Unpacking texlive-pictures (2021.20220204-1) ...
Selecting previously unselected package texlive-latex-extra.
Preparing to unpack .../50-texlive-latex-extra_2021.20220204-1_all.deb ...
Unpacking texlive-latex-extra (2021.20220204-1) ...
Selecting previously unselected package texlive-plain-generic.
Preparing to unpack .../51-texlive-plain-generic_2021.20220204-1_all.deb ...
Unpacking texlive-plain-generic (2021.20220204-1) ...
Selecting previously unselected package tipa.
Preparing to unpack .../52-tipa_2%3a1.3-21_all.deb ...
Unpacking tipa (2:1.3-21) ...
Selecting previously unselected package texlive-xetex.
Preparing to unpack .../53-texlive-xetex_2021.20220204-1_all.deb ...
Unpacking texlive-xetex (2021.20220204-1) ...
Setting up fonts-lato (2.0-2.1) ...
Setting up fonts-noto-mono (20201225-1build1) ...
Setting up libwoff1:amd64 (1.0.2-1build4) ...
Setting up libtexlua53:amd64 (2021.20210626.59705-1ubuntu0.2) ...
Setting up libijs-0.35:amd64 (0.35-15build2) ...
Setting up libtexluajit2:amd64 (2021.20210626.59705-1ubuntu0.2) ...
Setting up libfontbox-java (1:1.8.16-2) ...
Setting up rubygems-integration (1.18) ...
Setting up libzip-0-13:amd64 (0.13.72+dfsg.1-1.1) ...
Setting up fonts-urw-base35 (20200910-1) ...

```

```

Setting up poppler-data (0.4.11-1) ...
Setting up tex-common (6.17) ...
update-language: texlive-base not installed and configured, doing nothing!
Setting up libfontenc1:amd64 (1:1.1.4-1build3) ...
Setting up libjbig2dec0:amd64 (0.19-3build2) ...
Setting up libteckit0:amd64 (2.5.11+ds1-1) ...
Setting up libapache-pom-java (18-1) ...
Setting up ruby-net-telnet (0.1.1-2) ...
Setting up xfonts-encodings (1:1.0.5-0ubuntu2) ...
Setting up t1utils (1.41-4build2) ...
Setting up libidn12:amd64 (1.38-4ubuntu1) ...
Setting up fonts-texgyre (20180621-3.1) ...
Setting up libkpathsea6:amd64 (2021.20210626.59705-1ubuntu0.2) ...
Setting up ruby-webrick (1.7.0-3ubuntu0.1) ...
Setting up fonts-lmodern (2.004.5-6.1) ...
Setting up fonts-droid-fallback (1:6.0.1r16-1.1build1) ...
Setting up ruby-xmlrpc (0.3.2-1ubuntu0.1) ...
Setting up libsynchronet2:amd64 (2021.20210626.59705-1ubuntu0.2) ...
Setting up libgs9-common (9.55.0~dfsg1-0ubuntu5.10) ...
Setting up teckit (2.5.11+ds1-1) ...
Setting up libpdfbox-java (1:1.8.16-2) ...
Setting up libgs9:amd64 (9.55.0~dfsg1-0ubuntu5.10) ...
Setting up preview-latex-style (12.2-1ubuntu1) ...
Setting up libcommons-parent-java (43-1) ...
Setting up dvisvgm (2.13.1-1) ...
Setting up libcommons-logging-java (1.2-2) ...
Setting up xfonts-utils (1:7.7+6build2) ...
Setting up libptexenc1:amd64 (2021.20210626.59705-1ubuntu0.2) ...
Setting up texlive-binaries (2021.20210626.59705-1ubuntu0.2) ...
update-alternatives: using /usr/bin/xdvi-xaw to provide /usr/bin/xdvi.bin
(xdvi.bin) in auto mode
update-alternatives: using /usr/bin/bibtex.original to provide /usr/bin/bibtex
(bibtex) in auto mode
Setting up lmodern (2.004.5-6.1) ...
Setting up texlive-base (2021.20220204-1) ...
/usr/bin/ucfr
/usr/bin/ucfr
/usr/bin/ucfr
/usr/bin/ucfr
mktexlsr: Updating /var/lib/texmf/ls-R-TEXLIVEDIST...
mktexlsr: Updating /var/lib/texmf/ls-R-TEXMFMAIN...
mktexlsr: Updating /var/lib/texmf/ls-R...
mktexlsr: Done.
tl-paper: setting paper size for dvips to a4:
/var/lib/texmf/dvips/config/config-paper.ps
tl-paper: setting paper size for dvipdfmx to a4:
/var/lib/texmf/dvipdfmx/dvipdfmx-paper.cfg
tl-paper: setting paper size for xdvi to a4: /var/lib/texmf/xdvi/XDvi-paper

```

```

tl-paper: setting paper size for pdftex to a4: /var/lib/texmf/tex/generic/tex-
ini-files/pdftexconfig.tex
Setting up tex-gyre (20180621-3.1) ...
Setting up texlive-plain-generic (2021.20220204-1) ...
Setting up texlive-latex-base (2021.20220204-1) ...
Setting up texlive-latex-recommended (2021.20220204-1) ...
Setting up texlive-pictures (2021.20220204-1) ...
Setting up texlive-fonts-recommended (2021.20220204-1) ...
Setting up tipa (2:1.3-21) ...
Setting up texlive-latex-extra (2021.20220204-1) ...
Setting up texlive-xetex (2021.20220204-1) ...
Setting up rake (13.0.6-2) ...
Setting up libruby3.0:amd64 (3.0.2-7ubuntu2.8) ...
Setting up ruby3.0 (3.0.2-7ubuntu2.8) ...
Setting up ruby (1:3.0~exp1) ...
Setting up ruby-rubygems (3.3.5-2) ...
Processing triggers for man-db (2.10.2-1) ...
Processing triggers for fontconfig (2.13.1-4.2ubuntu5) ...
Processing triggers for libc-bin (2.35-0ubuntu3.4) ...
/sbin/ldconfig.real: /usr/local/lib/libtbbmalloc_proxy.so.2 is not a symbolic
link

/sbin/ldconfig.real: /usr/local/lib/libtbbbind.so.3 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbbind_2_5.so.3 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libumf.so.0 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libur_loader.so.0 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libhwloc.so.15 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libur_adapter_opencl.so.0 is not a symbolic
link

/sbin/ldconfig.real: /usr/local/lib/libtbb.so.12 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbmalloc.so.2 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbbind_2_0.so.3 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtcm.so.1 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libur_adapter_level_zero.so.0 is not a
symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtcm_debug.so.1 is not a symbolic link

```

Processing triggers for tex-common (6.17) ...
 Running updmap-sys. This may take some time... done.
 Running mktexlsr /var/lib/texmf ... done.
 Building format(s) --all.
 This may take some time... done.

```
[23]: # The rest of your code remains unchanged
from nbconvert import PDFExporter
import nbformat
# Montar Google Drive
from google.colab import drive
drive.mount('/content/drive')
# Load the Jupyter notebook file
notebook_path = '/content/drive/MyDrive/IA_ProyectoAgosto/Enero/
↳PintoA_EneroImagenes.ipynb' # Changed to the correct path to your .ipynb
↳file

with open(notebook_path, 'r') as notebook_file:
    notebook_content = nbformat.read(notebook_file, as_version=4)

# Export the notebook to PDF
pdf_exporter = PDFExporter()
pdf_data, resources = pdf_exporter.from_notebook_node(notebook_content)

# Save the PDF to a file
pdf_output_path = '/content/drive/MyDrive/IA_ProyectoAgosto/Enero/
↳PintoA_EneroImagenes.pdf'
with open(pdf_output_path, 'wb') as pdf_file:
    pdf_file.write(pdf_data)

pdf_output_path
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call `drive.mount("/content/drive", force_remount=True)`.

```
[23]: '/content/drive/MyDrive/IA_ProyectoAgosto/Enero/PintoA_EneroImagenes.pdf'
```

```
[ ]: import os
import numpy as np
import pandas as pd
from tensorflow.keras.preprocessing import image
from tensorflow.keras.applications.vgg16 import VGG16, preprocess_input
from tensorflow.keras.models import Model
from tensorflow.keras.preprocessing.image import load_img, img_to_array
from sklearn.preprocessing import LabelEncoder

# Configuración del modelo VGG16 preentrenado
```

```

base_model = VGG16(weights='imagenet', include_top=False, input_shape=(224,
↳224, 3))
model = Model(inputs=base_model.input, outputs=base_model.output)

# Función para extraer características de una imagen
def extract_features(img_path):
    img = load_img(img_path, target_size=(224, 224)) # Redimensionar la imagen
    img_array = img_to_array(img) # Convertir la imagen a un arreglo de numpy
    img_array = np.expand_dims(img_array, axis=0) # Añadir dimensión extra
↳para el batch
    img_array = preprocess_input(img_array) # Preprocesar para VGG16
    features = model.predict(img_array) # Obtener las características
    features = features.flatten() # Aplanar las características a un vector
    return features

    # Montar Google Drive (si aún no se ha montado)
from google.colab import drive
drive.mount('/content/drive')
# Suggested change: Verify the path and correct if necessary.
folder_path = '/content/drive/MyDrive/IA_ProyectoAgosto/datasetAffectNet/
↳AffectNet/train' # Corrected potential extra slash
import os

print(os.listdir(folder_path)) # This will print contents of the directory.
# Inicializar listas para las características y las etiquetas
features_list = []
labels_list = []

# Recorrer las carpetas
for folder in os.listdir(folder_path):
    folder_dir = os.path.join(folder_path, folder)
    if os.path.isdir(folder_dir):
        label = folder # Nombre de la carpeta será la etiqueta
        for img_name in os.listdir(folder_dir):
            img_path = os.path.join(folder_dir, img_name)
            if img_name.lower().endswith(('png', 'jpg', 'jpeg')):
                features = extract_features(img_path) # Extraer
↳características de la imagen
                features_list.append(features) # Añadir las características a
↳la lista
                labels_list.append(label) # Añadir la etiqueta a la lista

# Codificar las etiquetas
label_encoder = LabelEncoder()
encoded_labels = label_encoder.fit_transform(labels_list)

```



```

# Convertir a DataFrame
features_array = np.array(features_list)
df = pd.DataFrame(features_array)

# Añadir las etiquetas al DataFrame
df['Label'] = encoded_labels

# Guardar a un archivo CSV
csv_path = '/content/drive/MyDrive/IA_ProyectoAgosto/datasetAffectNet/
↳Aff_features.csv' # Reemplaza con la ruta donde deseas guardar el CSV
df.to_csv(csv_path, index=False)

print(f"Características y etiquetas guardadas en {csv_path}")

```